

DynamiQ: Accelerating Gradient Synchronization using Compressed Multi-hop All-reduce

Wenchen Han
University College London

Michael Mitzenmacher
Harvard University

Shay Vargaftik
VMware Research by Broadcom

Ran Ben Basat
University College London and Broadcom

Abstract

Multi-hop all-reduce is the de facto backbone of large model training. As the training scale increases, the network often becomes a bottleneck, motivating the reduction of the volume of transmitted data. Accordingly, recent systems have demonstrated significant acceleration of the training process using gradient quantization. However, these systems are not optimized for multi-hop aggregation, where entries are partially summed multiple times along their aggregation topology.

We present DynamiQ, a quantization framework that bridges the gap between quantization best practices and multi-hop aggregation. DynamiQ introduces novel techniques to better represent partial sums, co-designed with a decompress-accumulate-recompress fused kernel to facilitate fast execution.

We extend PyTorch DDP to support DynamiQ over NCCL P2P, and across different LLMs, tasks, and scales, we demonstrate consistent improvement of up to 34.2% over the best among state-of-the-art methods such as Omni-Reduce, THC, and emerging standards such as MXFP4, MXFP6, and MXFP8. Further, DynamiQ is the only evaluated method that consistently reaches near-baseline accuracy (e.g., 99.9% of the BF16 baseline) and does so while significantly accelerating the training.

CCS Concepts

• **Computer systems organization** → **Distributed architectures**; • **Computing methodologies** → **Machine learning**.

Keywords

Distributed training, all-reduce, gradient compression, quantization, algorithms, data-parallelism, large language models

ACM Reference Format:

Wenchen Han, Shay Vargaftik, Michael Mitzenmacher, and Ran Ben Basat. 2026. DynamiQ: Accelerating Gradient Synchronization using Compressed Multi-hop All-reduce. In *ACM SIGCOMM 2026 Conference (SIGCOMM '26)*, August 17–21, 2026, Denver, CO, USA. ACM, New York, NY, USA, 17 pages. <https://doi.org/10.1145/3789240.3829148>

SIGCOMM '26, Denver, CO, USA

© 2026 Copyright held by the owner/author(s).

This is the author's version of the work. It is posted here for your personal use. Not for redistribution. The definitive Version of Record was published in *ACM SIGCOMM 2026 Conference (SIGCOMM '26)*, August 17–21, 2026, Denver, CO, USA, <https://doi.org/10.1145/3789240.3829148>.

1 Introduction

Distributed data parallel (DDP) [30] is the standard paradigm for large language model (LLM) training and fine-tuning. Under this paradigm, the model is replicated across workers, each processing a different part of the data to compute a local gradient. These gradients are then synchronized (aggregated) via the network to obtain a global update. Gradient aggregation in LLM training commonly relies on multi-hop all-reduce schemes [4, 8, 57], such as ring [1] and butterfly [78]. With the growth of model sizes and the number of workers, gradient aggregation increasingly becomes a bottleneck [39, 71, 72, 76, 88, 89]. Recent practices of running multiple jobs in the same cluster, where jobs compete on network resources [25, 46], further intensify the bottleneck.

Gradient compression, which aims to reduce the volume of communicated gradient data, is therefore a natural and promising approach to accelerating gradient aggregation. Despite substantial prior work, we observe that state-of-the-art solutions [17, 36, 56, 68, 86, 87] typically consider the (single-hop) parameter-server architecture [55] where aggregation (after decompression) can be performed with higher precision without any bandwidth implications. In particular, they are not optimized for *multi-hop* all-reduce, where gradients are *partially summed* along their aggregation topology. In such a scheme, intermediate nodes face the choice of either recompressing the partial sum, thus degrading accuracy and eventually model performance, or increasing the number of bits used for its representation, leading to limited end-to-end speedups [15, 42]. As we show in Section 5, this limitation applies both to existing quantization and sparsification schemes (e.g., THC [56] and OmniReduce [36]), as well as recent microscaling floating-point (FP) formats [7, 68, 70].

In this paper, we focus on data-parallelism and introduce DYNAMIQ, a compression framework *tailored for multi-hop all-reduce* and is applicable to both pre-training and fine-tuning. DYNAMIQ minimizes the compression error of partial sums under a bandwidth constraint, utilizing a fused decompress-accumulate-recompress kernel to minimize memory bandwidth [3, 5, 85] and facilitate the overlap of compression with communication. The key to DYNAMIQ's superior accuracy-bandwidth tradeoff lies in its two-phase method of quantizing different coordinates with different numbers of bits, depending on their magnitude in *the aggregated gradient*.

Ideally, we would quantize each coordinate with a number of bits based on its magnitude. However, this imposes several challenges:

- Communicating each entry's quantized bit width would add prohibitive overhead.
- Arbitrary quantized widths can break byte alignment, preventing efficient fused kernels.

- Aperiodic widths harm memory coalescing. Moreover, offset metadata also significantly increases the memory and bandwidth overheads.
- Varying the bit allocation along the aggregation path requires repacking and hinders performance, as this cannot be done efficiently in a fused kernel.

Instead, our framework works as follows. DYNAMIQ joins consecutive entries into small *groups* (e.g., 16 entries), which are then further formed into *super-groups* (e.g., 16 groups). Groups and super-groups share metadata; groups have a shared scale parameter, and all entries in a super-group use the same bitwidth. This approach provides a good balance between bit allocation flexibility (from varying the group size) and metadata overhead. DYNAMIQ performs an initial lightweight all-reduce call to collect necessary statistics about super-groups. This enables all workers to agree on the bit allocation, which then remains fixed throughout the aggregation. Further, all workers then reorder the super-groups according to their bit allocation, which enables fused kernel invocations on sequential data in the main all-reduce.

To further optimize the bandwidth-accuracy tradeoff, DYNAMIQ uses advanced quantization techniques, including:

- *Non-uniform quantization* — DYNAMIQ normalizes the data and uses a pre-determined non-uniform set of quantization values that optimizes the per-entry multiplicative error. Intuitively, this is achieved by using more quantization values that are closer to zero than larger ones, akin to floating-point formats. Specifically, we use the choice of quantization values proposed by [34].
- *Negative correlation across workers* — DYNAMIQ uses correlated rounding so the errors are more likely cancel out [75]. Intuitively, DYNAMIQ uses shared randomness to increase the probability that if one worker rounds upwards, another will round down, lowering the aggregation error.

We integrate DYNAMIQ into PyTorch DDP [57] via a communication hook that runs over NCCL P2P [11]. We evaluate DYNAMIQ across different LLM fine-tuning workloads (BERT-large [31] Masked LM, LLaMA-1B [35] chat & MMLU [44], Gemma-1B [77] Chat) and all-reduce topologies (ring and butterfly). DYNAMIQ improves the time-to-accuracy by up to 34.2% over the best among OmniReduce [36], THC [56] and modern floating point (FP) formats (MXFP4/MXFP6/MXFP8) [7]. In several settings, DYNAMIQ is the only method that reaches near-baseline accuracy (i.e., 99.9% final accuracy relative to BF16) in all our workloads while accelerating training by 40.8% compared with BF16. In simulated setups, we show that DYNAMIQ scales well to large DP-dimensions. For example, with a DP-dimension of 8192, DYNAMIQ with $\bar{b} = 6$ bits per coordinate is more accurate than MXFP8 ($\bar{b} = 8.5$ per coordinate) by a large margin. Our code is available at [41].

2 Background

In this section, we provide necessary background on quantization. Quantization is the process of mapping a continuous or high-precision set of values to a smaller, discrete set of values, essentially reducing the number of bits used to represent a number. In the context of distributed gradient-based training frameworks (e.g., using a distributed SGD, ADAM [51], or AdamW [59] for LLMs), at each training round, gradients from different workers must be

aggregated to compute a global gradient. Thus, employing gradient quantization at the workers reduces communication. The challenge, however, is to make the quantization accurate and fast, so that it takes less time to reach the desired model accuracy, a metric known to *time-to-accuracy*.

2.1 Unbiased quantization

An important property of gradient quantization is being *unbiased*. Namely, given a gradient $X \in \mathbb{R}^d$ and its quantized estimate $\hat{X}$, we would like to have $\mathbb{E}[\hat{X}] = X$.

Intuitively, unbiasedness is desirable when averaging, as when some values are rounded up and others are rounded down, the errors cancel out in expectation. Critically, under mild conditions, unbiased quantization ensures training convergence [16].

A fundamental method for unbiased quantization is *stochastic quantization* (SQ). In SQ, given a scalar $x \in \mathbb{R}$ and two quantization values $x_\downarrow, x_\uparrow$ where $x \in [x_\downarrow, x_\uparrow]$, we obtain $\mathbb{E}[\hat{x}] = x$ by setting

$$\hat{x} = \begin{cases} x_\uparrow & \text{w.p. } \frac{x - x_\downarrow}{x_\uparrow - x_\downarrow} \\ x_\downarrow & \text{otherwise} \end{cases}$$

More generally, when quantizing a vector $X \in \mathbb{R}^d$ using a set of quantization values $Q \subset \mathbb{R}^d$, for each $x \in X$, we denote by $x_\downarrow = \max\{q \in Q \mid q \leq x\}$ and $x_\uparrow = \min\{q \in Q \mid q \geq x\}$ its two closest values, and apply SQ as described above. Thus, each quantized value $\hat{x} \in Q$ can be represented using $\log_2 |Q|$ bits using its quantization value index (e.g., for 4 bits per coordinate, we can use $|Q| = 16$).

2.2 Grouped quantization

To dequantize a quantized vector, the receiver must know the set Q . While many previous works select a single Q for an entire all-reduce call, we opt to choose one set for each *group* (a consecutive sequence of e.g., 16 entries) as it allows optimizing Q for the specific entries.

There are two main reasons why a per-group choice of Q is more accurate. First, gradients often exhibit *spatial locality*, where nearby entries tend to have similar magnitudes. Grouping, therefore, tailors Q to a small range of values. Second, the gradient distribution itself is skewed, with a small number of coordinates (outliers) that can be orders of magnitude larger than others. These outliers have a disproportionate effect on the set Q and the effectiveness of quantization. Using per-group values Q with small groups reduces the overall effect of these outliers.

Intuitively, due to the above reasons, the smaller the groups are, the more accurate the quantization is, as we tailor Q (and the size of Q) for each specific group. However, since each group has an overhead (i.e., the encoding of its Q), having too many small groups inflates the required bandwidth, defeating the purpose of quantization. To reduce the encoding overheads, one can use super-groups (e.g., of 16 consecutive groups each) to share some of the metadata [9].

We conduct an experiment to exemplify spatial locality and skewness. To do so, we compare the distribution of group and super-group norms in the original gradient to these distributions after randomly shuffling the entries. Intuitively, if there is no spatial locality, the distributions are likely to be similar. We analyze the first gradient of fine-tuning LLaMA 1B for MMLU [44] and Gemma 1B for Ultrachat [32] (the complete experimental details appear in Section 5). We show the results for groups of size 16 in

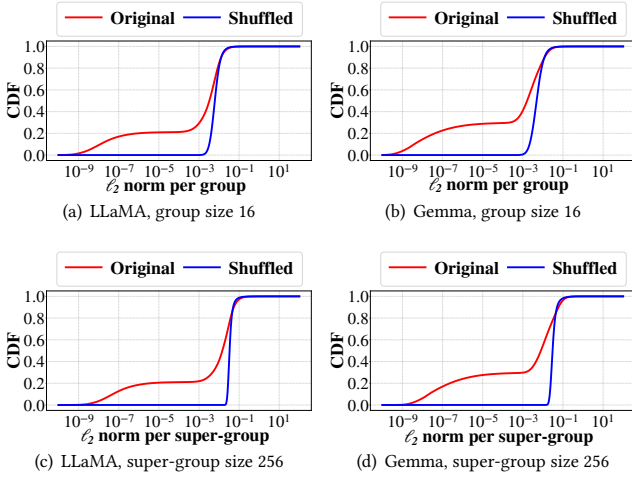


Figure 1: ℓ_2 norm distributions of the gradients and their random shuffle for groups of size 16 and super-groups of size 256. The detailed experimental setups appear in Section 5.

Figures 1(a)-1(b) and for a sequence of 16 groups (i.e., super-groups) in Figures 1(c)-1(d). This spatial locality yields a significant fraction (e.g., about 20% in LLaMA and 30% in Gemma) of super-groups with a norm that is orders of magnitude smaller than the median, highlighting the opportunity for variable bitwidth allocation.

2.3 Non-uniform quantization

Given an input $X \in \mathbb{R}^d$, the common choice of Q is to place $|Q|$ uniformly spaced quantization values in the range $[\min X, \max X]$ (e.g., QSGD [16] and Uniform-THC [56]). To optimize the accuracy for $|Q|$ quantization values, we can place them non-uniformly.

For example, consider $X = (-1, 1/2, 1)$; if we want $|Q| = 3$, uniform SQ would use $Q = \{-1, 0, 1\}$ resulting in a Mean Squared Error (MSE) of $\sum_{x \in X} \mathbb{E}[(\hat{x} - x)^2] = 1/4$. In contrast, picking the non-uniform $Q = \{-1, 1/2, 1\}$ results in an MSE of 0. Generally, the MSE of non-uniform SQ can be asymptotically lower, e.g., for $X = (-1, 1/2, \dots, 1/2, 1)$, i.e., where $1/2$ repeats $d-2$ times. Here, the MSE of uniform SQ is $\Omega(d)$ while the non-uniform SQ is accurate.

2.4 Negative correlation

Intuitively, we can take the idea of unbiased quantization a step forward by explicitly ‘encouraging’ errors to cancel out [75]. This is achieved by using shared randomness between different workers, which is facilitated by them sharing a pseudo-random number generator seed. Using shared randomness is a common practice in quantization works (e.g., [18–22]).

For example, consider the simple case where two workers hold numbers $x_1, x_2 \in [0, 1]$ that need to be quantized into 1 bit each, with the goal of estimating $x_1 + x_2$. A standard approach is to have independent randomness where workers generate uniform random variables $u_1, u_2 \sim \mathcal{U}[0, 1]$ and quantize

$$\hat{x}_1 = \begin{cases} 1 & \text{if } u_1 \leq x_1 \\ 0 & \text{otherwise} \end{cases}, \quad \hat{x}_2 = \begin{cases} 1 & \text{if } u_2 \leq x_2 \\ 0 & \text{otherwise} \end{cases}.$$

To leverage negative correlation, we use the shared randomness to set $u_2 = 1 - u_1$; i.e., both workers generate the same $u_1 \sim \mathcal{U}[0, 1]$

but use it in a different way to increase the chances that they round in the opposite direction. For example, if $x_1 = x_2 = 1/2$, the independent randomness variance is $\text{Var}[\hat{x}_1 + \hat{x}_2] = 1/2$ while the negative correlation’s variance is 0. More generally, the variance (for any input x_1, x_2) of the negative correlation approach is at most $1/4$, i.e., improving the worst-case variance by a factor of 2.

3 The DYNAMIQ Framework

Figure 2 illustrates an overview of the framework.

In the first stage, each worker partitions the gradient into super-groups of S entries, and computes the metadata per super-group (Fig. 2(a)). Next, this local metadata is aggregated by an initial all-reduce call (Fig. 2(b)) that is lightweight since it contains only the super-group means and sum of ℓ_2 norms, so the volume typically less than 1% of the original gradient. We then perform per-super-group normalization and allocate variable bitwidths for different super-groups. We reorder the gradient such that super-groups with the same bitwidth appear consecutively (Fig. 2(c)), and perform the main all-reduce (Figs. 2(d) and 2(e)). Finally, we post-process the aggregated data by reordering the super-groups to their original position and adding their mean value back to obtain the synced gradient (Fig. 2(f)). We also propose and adopt several techniques to reduce the compression error and implement our algorithm using fused kernels to minimize the computational overhead.

3.1 Obtaining super-group statistics

We formalize the first two stages (Fig. 2(a) and Fig. 2(b)). Assume a setup with n workers. Let $X_{i,j}$ be the j ’th super-group of worker i . For each i, j , the i ’th worker first computes the mean given by $\mu_{i,j} = \sum_{x \in X_{i,j}} x / |X_{i,j}|$ and squared ℓ_2 norm given by $F_{i,j} = \sum_{x \in X_{i,j}} x^2$. DYNAMIQ then uses an initial all-reduce stage to aggregate these values. At the end of this stage, for each super-group j , all workers have the global mean μ_j and sum-of-squared-norms F_j for that super-group. Formally,

$$\mu_j = \frac{1}{n} \cdot \sum_{i=1}^n \mu_{i,j}, \quad F_j = \sum_{i=1}^n F_{i,j}.$$

As illustrated in Figure 2(c), once these values are obtained, each worker normalizes their data by subtracting μ_j from each entry in super-group j , making it zero-mean, and then uses F_j values to determine the bitwidth and reorder the data as we explain next.

3.2 Determining super-group bitwidths

Due to the skewed distribution of the gradients (Figure 1), allocating more bits to super-groups with larger norms could substantially decrease the quantization error. We use variable bitwidth allocation to minimize the quantization error and respect any given bandwidth constraint. In particular, DYNAMIQ accepts a parameter $\bar{b}$ that denotes the average per-coordinate bit-budget. That is, for any throughput optimal primitive (e.g., Ring, Butterfly), the total number of bits to be sent by the network is $\frac{2(n-1)}{n} \bar{b}$ per coordinate per worker, where n is the number of workers. This includes both the quantized data and any additional metadata (e.g., scales). (Note $\bar{b}$ must be large enough for the metadata and 1 bit per entry.)

To allow efficient bit packing, we limit the possible quantized bitwidths to powers of 2, namely, 1, 2, 4, 8, and 16. This also has the

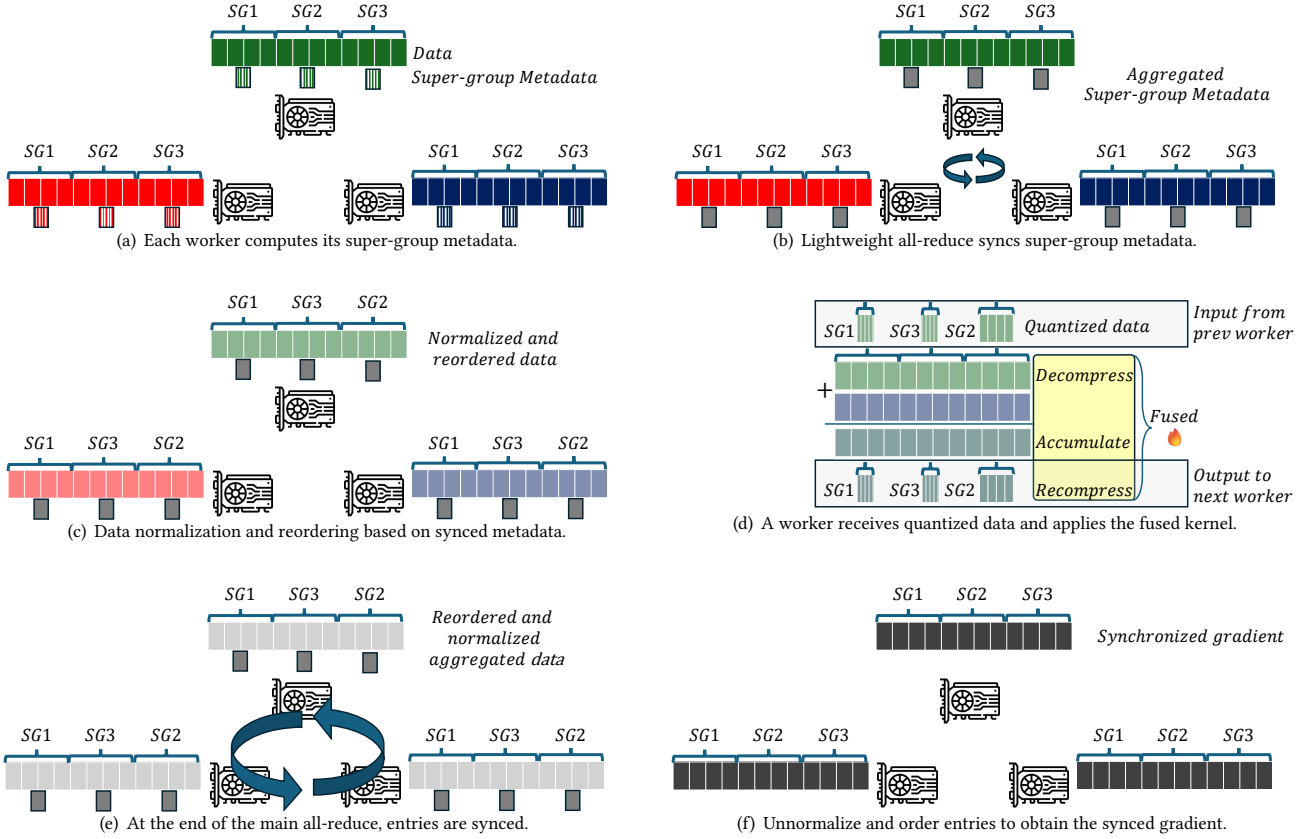


Figure 2: The DYNAMIQ workflow: (a) workers first compute the metadata (mean and ℓ_2 norm) for each of their super-groups; (b) a lightweight all-reduce call aggregated the metadata such that all workers know that global super-group means and sum of ℓ_2 norms; (c) based on the aggregated metadata, each worker normalizes each super-group by subtracting its global mean and reorders the super-groups based on their bit width which is based on the ℓ_2 norms. Notice that in this example, SG3 has lower bit width than SG2 and thus their places are swapped; (d) illustrates how the blue worker operates during the main all reduce. It invokes the fused kernel to first decompress the received compressed partial sums data from the green worker, accumulates its local data, and recompresses the result before sending it to the red worker; (e) after the main all-reduce terminates, all workers have the same aggregated sums; (f) each worker adds back the global mean of each super-group and orders the data back to obtain the synced gradient.

benefit of simplifying finding a performant variable quantization. As the choice of 16 bits corresponds to uncompressed values, this generalizes existing approaches where some entries (such as top- k /outliers) are encoded accurately while others are quantized to a smaller number of bits [23].

We now describe our fast heuristic approach to bandwidth partitioning to the different super-groups. For a given set of allowed bitwidths (e.g., $W = \{1, 2, 4, 8, 16\}$), we use *thresholds* $T_{a,b}$ (dashed lines in Figure 3), where a, b are consecutive in W , to denote the boundary of the F_j values with the same allocation. Intuitively, the MSE of quantizing a set is proportional to its squared norm, and thus the F_j values serve as proxy for the expected error of the j 'th super-group. For ease of presentation, in this section we use the above W , although the technique is general.

Let $T_{0,1} = 0$ and $T_{16,32} = \infty$ for convenience. Then, all entries within super-groups with $F_j \in [T_{a,b}, T_{b,c})$ are quantized to b bits. Next, we derive the relations between the thresholds. Suppose we start from a given set of thresholds $\{T_{a,b}\}$ and want to increase

the bandwidth in a way that best reduces the MSE. This can be achieved by lowering a selected threshold $\{T_{a,b}\}$, increasing the quantized bit width of some super-group from a to b bits per entry.

The intuition behind our approach is based on a simple worst-case analysis. Consider an example where Q includes two quantization values 0, 1 and an entry $x \in [0, 1]$. The worst-case variance for $\hat{x}$ is when $x = 1/2$, which yields $\text{Var}[\hat{x}] = 1/4$. Now, suppose we increase the quantization bit width by 1 bit per entry (i.e., we double the size of Q). In that case, we can place a quantization value between every two consecutive values in Q , including between 0 and 1. If this extra quantization value is at $1/2$, the worst-case becomes when $x = 1/4$, yielding $\text{Var}[\hat{x}] = 1/16$, i.e., a $4\times$ reduction. This reduction generalizes to any two consecutive quantization values $x_{\downarrow}, x_{\uparrow} \in Q$, i.e., for each additional bit, we can reduce the worst-case MSE of the entire vector by $4\times$.

Suppose we decrease $T_{a,b}$ just enough that a single super-group j will have its entries encoded by b bits instead of a . The above intuition suggests that each added bit decreases the MSE roughly $4\times$.

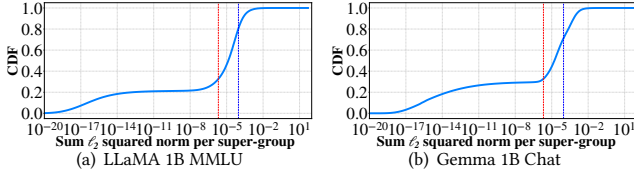


Figure 3: The CDF distribution of F_j , summed ℓ_2 squared norm per super-group across workers. The vertical dashed lines are thresholds for our variable bitwidth allocation algorithms, where super-groups with larger ℓ_2 norms are assigned more bits in one of 2, 4, or 8 bits.

If the super-group previously had an MSE proportional to $T_{a,b} \cdot 4^{-a}$, then its MSE lowers by roughly proportionally to $T_{a,b} \cdot (4^{-a} - 4^{-b})$ while increasing the bandwidth by $b - a$ bits per entry in this super-group. We therefore estimate the *per-bit benefit* of this action by $\frac{T_{a,b} \cdot (4^{b-a} - 1)}{4^{b-a} \cdot (b-a)}$. For example, if $a = 1, b = 2$, we lower the MSE by about $T_{1,2} \cdot 3/16$ with a per-bit benefit of $T_{1,2} \cdot 3/16$. Similarly, if $a = 2, b = 4$, we lower the MSE about $T_{2,4} \cdot 15/256$ and the per-bit benefit is $T_{2,4} \cdot 15/512$ while the per-bit benefit of lowering $T_{4,8}$ is $255/4^9$. To optimize the thresholds selection, we require that the per-bit benefit of increasing all thresholds is roughly the same (e.g., $T_{1,2} \cdot 3/16 = T_{2,4} \cdot 15/256$), yielding:

$$T_{1,2} = 5/32 \cdot T_{2,4}, \quad T_{2,4} = 17/512 \cdot T_{4,8}, \quad T_{4,8} = 257/2^{17} \cdot T_{8,16}.$$

Notice that this gives us $|W| - 1$ constraints, leaving a single degree of freedom (e.g., by selecting $T_{1,2}$, the rest are determined). Accordingly, we search for the value of $T_{1,2}$ and determine the other thresholds by the above formula, such that the desired bandwidth constraint $\bar{b}$ is met.

Since we are interested in minimizing the computational overhead of finding the above threshold, for the practical case where at most three permissible bitwidths are used (e.g., we use $W = \{2, 4, 8\}$ in our implementation), we develop a fast binary-search-based solution to determine the thresholds in Section A.

3.3 DYNAMIQ's quantization

We now describe DYNAMIQ's quantization algorithm. The algorithm is used to compress the data of the first chunk along the aggregation topology, as well as for decompressing and recompressing partial sums, as illustrated in Figure 2(d).

Non-uniform quantization. Motivated by Section 2.2, DYNAMIQ uses grouped quantization where a given group G is quantized using b bits. Each group is also associated with metadata, namely, a scaling parameter sf , which is used as explained below. With b bits per entry, we can represent each entry using a sign bit and a representation in $\{0, \dots, 2^{b-1} - 1\}$. We hereafter assume that all quantization values $q \in Q$ are non-negative as we encode the sign bit separately. We choose the values of Q non-uniformly, similarly to [34]. Specifically, denoting $f(\epsilon, r) = \frac{(1+2\epsilon^2)^r - 1}{(1+2\epsilon^2)^{2^{b-1}-1} - 1}$, for a parameter $\epsilon > 0$, we use

$$Q = \{f(\epsilon, r) \mid r \in \{0, \dots, 2^{b-1} - 1\}\}.$$

The result is some $Q \subset [0, 1]$, where ϵ affects how non-uniform the quantization values are. Intuitively, when $\epsilon \approx 0$, Q is roughly

uniformly partitioned in $[0, 1]$; a larger ϵ yields more quantization values close to zero and fewer large ones.

Note that as the entries in G can be arbitrary BF16 values, we need to normalize them to $[0, 1]$ before we can stochastically quantize to Q ; this is achieved by dividing each entry x by $\max |G| \triangleq \max \{|x| \mid x \in G\}$ and encoding the sign of x separately.

Hierarchical quantization. Next, the sender represents the group using a scaling factor sf and a per-entry tuple (r, ζ) , where r is the representation and ζ is the sign bit. In turn, the receiver estimates the entry as $\zeta \cdot f(\epsilon, r) \cdot sf$.

The natural choice is to set the scaling factor of the group G to $sf_G = \max \{|x| \mid x \in G\}$. However, this would require transmitting sf_G in high precision (e.g., 16 bits), incurring a significant bandwidth overhead when the group size s is small.

Instead, DYNAMIQ optimizes the accuracy-bandwidth tradeoff by quantizing the scaling factors within a super-group, a method known as *hierarchical quantization* [9]. Namely, let $\mathcal{G}$ be a super-group, and let $sf_{\mathcal{G}} = \max |G|$ equal the largest absolute value of an entry in $\mathcal{G}$, i.e., $\max \{|x| \mid x \in \mathcal{G}\}$. We encode $sf_{\mathcal{G}}$ in half-precision for the entire super-group, and quantize each individual group's scaling factor using uniform stochastic quantization such that $\mathbb{E}[sf_G] = \max |G|$. For example, we can represent sf_G using a UINT8 representation $r_G \in \{0, \dots, 255\}$, decoded as $sf_G = r_G \cdot sf_{\mathcal{G}}/255$.

An essential property of our hierarchical quantization is that the estimates of individual entries remain unbiased. Consider a specific entry $x \in G$. It is first normalized to $x' = x/\max |G|$, which is then stochastically quantized to $\hat{x}' \in Q$. As explained above, the scale of the group itself is then stochastically quantized to

$$sf_G \in \{r_G \cdot sf_{\mathcal{G}}/255 \mid r_G \in \{0, \dots, 255\}\}.$$

The estimated value of x is therefore: $\hat{x} = \hat{x}' \cdot sf_G$, which, due to the independence of the randomness used in the two quantization steps (of x' and of sf_G), satisfies:

$$\mathbb{E}[\hat{x}] = \mathbb{E}[\hat{x}' \cdot sf_G] = \mathbb{E}[\hat{x}'] \cdot \mathbb{E}[sf_G] = (x/\max |G|) \cdot \max |G| = x.$$

That is, even though quantizing sf_G simultaneously scales all entries in G within the same scaling factor, the individual entries' unbiasedness is retained.

Correlated rounding. We now explain how to leverage negative correlation (Section 2.4) in the stochastic quantization of the entries. Intuitively, we want to increase the likelihood that if a given worker quantizes a specific partial sum upwards, another will quantize down, so that the overall result is closer to the true sum.

Formally, let p_i be the probability of rounding up a given recompressed partial sum at worker i . As exemplified in Section 2.4, stochastic rounding is commonly implemented by drawing a uniform random variable $u_i \sim \mathcal{U}[0, 1]$ and rounding up if $u_i < p_i$ and rounding down otherwise. Using Suresh et al. [75] correlated sampling method, instead of drawing u_i independently at random for different workers, we set

$$u_i = \frac{\pi_i + \gamma_i}{n}.$$

Here, $\pi = \{\pi_i\}$ is a random permutation of $0, \dots, n-1$ and $\gamma_i \sim \mathcal{U}[0, 1]$. Importantly, π is implicitly agreed upon by all workers as it is independently generated using the same pseudo-random number generator and is not communicated.

Each u_i is still uniformly distributed, but now correlated across workers. That is, in every interval $[0, \frac{1}{n}), \dots, [\frac{n-1}{n}, 1)$ exactly one worker's u_i falls inside. Intuitively, if u_i from one worker falls within $[0, \frac{1}{n})$ so that worker i rounds up with high probability, there will be another worker i' such that $U_{i'} \in [\frac{n-1}{n}, 1)$ and worker i' rounds down with high probability, canceling the error.

3.4 Main all-reduce

DYNAMIQ's main all-reduce phase follows standard communication patterns such as ring [1] and butterfly [78], but with compressed transfers. We next explain DYNAMIQ's two all-reduce components.

Reduce-scatter. The n workers split their gradients into n chunks, where $C_{i,j}$ is the i 'th chunk of the j 'th worker. For each index $i \in \{0, \dots, n-1\}$, chunks $\{C_{i,j} \mid j \in \{0, \dots, n-1\}\}$ are aggregated, in parallel with chunk sets of other i 's. For each such i , the reduce-scatter topology is an in-arborescence, i.e., a tree where all edges point towards a single sink. For example, on a ring all-reduce, the aggregation topology for a single chunk is simply a path, while for butterfly we visualize the topology in Figure 13 in Section B.¹

The aggregation works as follows: Leaf nodes (which receive no external messages for a specific chunk index i) transmit their compressed local chunk to the next node in the topology. Internal nodes serve as intermediaries that decompress and aggregate partial sums (including their local chunk), recompress, and send to the next node, while the sink node for the chunk terminates its reduce-scatter phase by decompressing and adding its local chunk.

All-gather. Upon the completion of the reduce-scatter phase, the all-gather phase commences, during which the sinks broadcast the aggregated sums to all other workers.

Fused kernels. We now describe how we optimize the all-reduce (illustrated in Figure 2(d)). DYNAMIQ employs four distinct types of fused kernels determined by the accumulation state and node type. The first kernel is used to compress the chunk entries at the leaf nodes. Internal nodes then use a decompress-accumulate kernel when they have received partial sums from all but the last of their parent nodes. When receiving the partial sum of the last parent, they apply a decompress-accumulate-recompress kernel to also get the sum ready for the next transmission.

The aggregated compressed chunk sums then continue to the all-gather phase, where they are broadcast. Whenever a node receives a compressed sum, it invokes the decompress kernel (ending in Figure 2(e)). The operation concludes with a final reconstruction step (Figure 2(f)) where entries are restored to their original order and unnormalized by adding back the mean value, which was subtracted during the initial all-reduce step, to recover the final output.

3.5 Communication and runtime overhead

We summarize DYNAMIQ's communication and runtime overheads.

Communication overhead. DYNAMIQ incurs communication in two all-reduce stages. First, in the lightweight all-reduce stage, let C and C_s be the group and super-group sizes. For each super-group $\mathcal{G}$, DYNAMIQ computes exactly two BF16 numbers via the reduction, $\mu_{\mathcal{G}}$ and $F_{\mathcal{G}}$. Therefore, this stage communicates $32/C_s$ bits per

coordinate. Second, in the main all-reduce stage, as explained in Section 3.2, DYNAMIQ deducts the bandwidth used by the lightweight stage from the overall communication budget. Thus, the main stage uses $\bar{b} - 32/C_s$ bits per coordinate. For example, for $\bar{b} = 5$ and $C_s = 256$, the first stage uses $32/C_s = 0.125$, leaving $\bar{b} - 0.125 = 4.875$ bits per coordinate for the second (main) stage for both the quantized representations and group and super-group scales. For groups of size $C = 16$, the group's scale takes $8/C = 0.5$ bits per coordinate and the super-groups' scale takes additional $16/C_s = 1/16$ bits per coordinate. This leaves $4.875 - 0.5 - 1/16 = 4.3125$ bits on average for each quantized coordinate's representation. This bit budget is divided among super-groups with, e.g., 2, 4, or 8 bitwidths (as detailed in Section 3.2) such that the resulting average is 4.3125. The reordering stage incurs no communication, since it only permutes super-groups locally into a pre-allocated tensor in monotonically increasing bitwidth order.

Compute overhead. All DYNAMIQ operations are linear in the gradient size and rely on sequential memory accesses to HBM. The lightweight stage computes $\mu_{\mathcal{G}}$ and $F_{\mathcal{G}}$ for all super-groups using a single kernel launch with one memory access per entry. The reordering stage performs a single read and write per entry, with only a small additional overhead for maintaining the super-group indices. The main decompress-accumulate and decompress-accumulate-recompress operations are each implemented as a single fused kernel. Each value is reconstructed on the fly from its quantization bin and scale, accumulated, and requantized without materializing intermediate tensors. Correlated rounding adds only local arithmetic and does not require additional memory accesses.

4 Implementation

We implement the DynamiQ prototype atop PyTorch DDP [57] with NCCL [11] P2P. Our code is available at Github [41]. As mentioned, our prototype centers on four CUDA kernels:

- (1) `DynamiQ_compress( $t$ )` compresses a gradient chunk t at a leaf node in the aggregation topology.
- (2) `DynamiQ_decompress( $ct$ )` decompresses a compressed gradient chunk ct at the all-gather stage of all-reduce.
- (3) `DynamiQ_decompress_accumulate_recompress( $ct, t$ )` in non-leaf nodes fuses decompressing ct , accumulating it with t , and recompressing the sum.
- (4) `DynamiQ_decompress_accumulate( $ct, t$ )` in intermediate hops executes decompression of ct and accumulating the result with t (without recompression).

Efficient fused-kernel CUDA implementation. GPUs are typically memory-bound for elementwise operations [3, 5, 40], which means that the compression overhead is mainly determined by global memory transactions. Our design, therefore, minimizes this overhead by using fused kernels.

`DynamiQ_decompress_accumulate_recompress`, for example, fuses these three operations. The intermediate results are stored in registers, avoiding global memory accesses. As Table 2 shows, this significantly reduces memory traffic, making DynamiQ's involved logic computationally lightweight.

The super-group reordering allows GPU kernels to receive a sequence of streams of uniform bitwidth entries, enabling efficient

¹Note that as the different i 's are aggregated in parallel; e.g., in ring all-reduce, each node acts both as a sender and a receiver, and the total communication pattern forms a cycle.

memory addressing and coalescing. To share the maximum per-group gradient value for scaling, we use the classic parallel maximum reduction algorithm in CUDA. We use powers of two for the group size and super-group size to allow more effective threaded memory access and execution.

The DDP communication hook and RDMA engine. We build the communication hook from P2P primitives, using NCCL within a node and RDMA across nodes. This enables all-reduce topologies not natively supported by NCCL collectives, including butterfly. Also, our RDMA engine overlaps compression with communication using blockwise pipelining. As soon as a gradient block completes `decompress_accumulate_recompress`, its RDMA transfer begins while subsequent blocks are processed (we use a block size of 8 MB in our experiments).

5 Evaluation

In this section, we present an end-to-end testbed evaluation of DYNAMIQ across four LLM full-parameter supervised fine-tuning workloads (that produce dense gradients) and compare it with state-of-the-art gradient compression schemes. We first use the ring all-reduce topology and evaluate performance both in isolation (Section 5.1) and over a shared network (Section 5.2). Then, to demonstrate the applicability of DYNAMIQ to different topologies, we then consider butterfly all-reduce (Section 5.3).

Testbed. Our testbed consists of four servers running CentOS. Each server is equipped with two NVIDIA RTX A6000 GPUs, each with 48 GB of GDDR6 memory and 768 GB/s of memory bandwidth. The two GPUs are connected through NVLink (NV4). Each server also has two 100 Gb/s NVIDIA Mellanox ConnectX-6 NICs, of which we use one in our experiments. Each server contains two 16-core AMD EPYC 7313 CPUs and 512 GB of host memory. Each GPU is connected to the host through PCIe 4.0 x16, providing approximately 32 GB/s of bandwidth per direction.

Workloads. We evaluate DYNAMIQ on four diverse training (fine-tuning) tasks representative of distinct model families and paradigms: standard masked language modeling with BERT-large [31] on Wikitext-103 [61]; decoder-based instruction tuning with Gemma 1B [77] on UltraChat [32]; reasoning with LLaMA 1B [35] on both UltraChat and MMLU [44].

Parameter setup. Table 1 summarizes the batch size (in terms of tokens and sequences) and learning rate configurations we use in our experiments. For Wikitext and UltraChat, following common practice, we truncate and pack tokens into fixed-length sequences (potentially merging consecutive samples). For learning rate scheduling, we use the standard `torch.optim.lr_scheduler.LinearLR` scheduler with parameters detailed in the table.

Baseline compression schemes. We compare DYNAMIQ against standard uncompressed BFloat16 [2] (BF16) format and 5 state-of-the-art gradient compression schemes covering both traditional and emerging standards: adaptive sparsification via OmniReduce (OR) [36], optimized fixed-point quantization via THC [56] and emerging microscaling formats (MXFP8, MXFP6 and MXFP4) [70].

THC and OmniReduce were originally designed for the parameter-server architecture [55] and are thus not optimized for multi-hop all-reduce. To ensure a fair comparison, we adapt them as follows.

For THC, directly summing quantized gradients via multi-hop all-reduce leads to catastrophic overflows given limited bits b . Consequently, following the original paper, we compress local gradients into $q = 4$ -bit integers and allocate $b = 8$ bits per coordinate for aggregation. For OmniReduce, we allocate $b = 8$ bits and employ its chunked Top- k compression variant. Since local Top- k chunk indices differ across workers, we first aggregate indices appearing in at least one worker and then transmit the union. The challenge lies in dynamically determining k such that the union of local Top- k chunks equals K ; we address this with a heuristic algorithm described in Appendix C.

For the microscaling formats, we use E4M3, E3M2, and E2M1 for MXFP8, MXFP6, and MXFP4, respectively. For all formats, the chunk size is 32 and the per-chunk shared scale is BF16. For summation arithmetic and overflow and underflow handling, we follow the implementation of FP8-LM [68]. We also dynamically adjust the per-chunk scales prior to the main all-reduce stage across training rounds to minimize overflows and underflows as described in Section C. Finally, since MXFP4 and MXFP6 are not natively supported by our testbed GPUs, we report their best-case (i.e., lower-bound) TTA estimates. This is done by decoupling accuracy (which is software-based) and timing (we transmit the equivalent traffic without performing any compute) measurements.

DYNAMIQ’s configuration. Unless otherwise stated, we set the group size as $s = 16$ coordinates and the super-group size as $S = 256$ coordinates (i.e., 16 consecutive groups). As explained in Section 3.3, per-group scaling parameters are quantized into UINT8 and per-super-group scaling parameters are kept as BF16.

We use $W = \{2, 4, 8\}$ and the approximation algorithm of Section A to determine the bit allocation. Unless otherwise noted, we fix the overall budget at $b = 5$ bits per coordinate, which we demonstrate to achieve the best end-to-end performance in our ablation study (see Figure 7).

Metrics. We evaluate the end-to-end performance using the well-established time-to-accuracy (TTA) metric and pay special attention to the final accuracy. Namely, while TTA tracks evaluation accuracy² as a function of wall-clock time, final accuracy is recorded once the model converges. We also benchmark throughput (in rounds per second) and quantization error to assess the speed and quality of our quantization compared to other schemes. For the latter, we employ the vector normalized mean squared error (vNMSE) metric [23, 33, 49, 56, 80, 81], defined as $\mathbb{E} \left[\frac{\|X - \hat{X}\|^2}{\|X\|^2} \right]$.

Workload	BERT-large MaskedLM	LLaMA 1B Chat	Gemma 1B Chat	LLaMA 1B MMLU
Tokens per batch	2048	3000	3000	~ 1600
Batch size	1	1	1	4
Initial LR	5×10^{-5}	2×10^{-5}	1.4×10^{-5}	6×10^{-6}
Linear LR end factors	1/16	1/8	1/8	1/8
Linear LR total iters (Epochs)	15	2	2	2
Total iters (Epochs)	21	3	3	3

Table 1: Configurations of the average number of tokens and the learning rate schedules in our workloads.

²Here we use the general term “accuracy” to refer to the respective LLM evaluation metric, including perplexity for MaskedLM and CausalLM for UltraChat, and classification accuracy for multiple choice question answering.

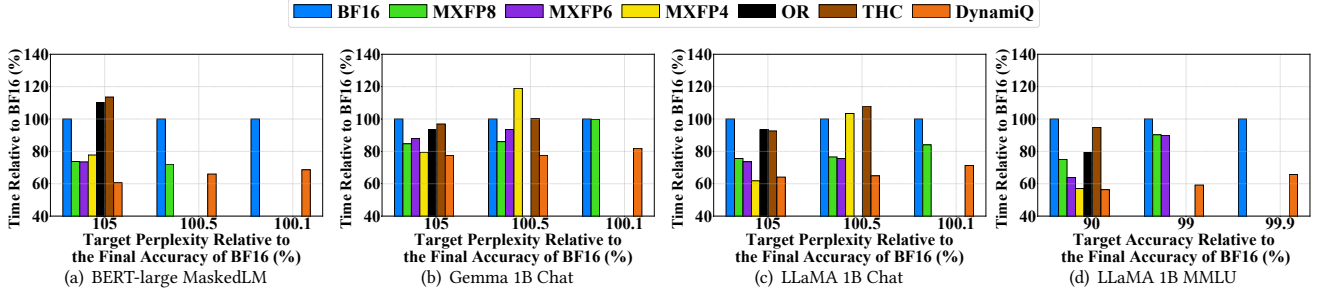


Figure 4: Time-to-target perplexity and accuracy for training (fine-tuning) LLMs on 8-GPU/4-worker testbed using ring all-reduce. We measure the time required relative to BF16 (lower is better) to reach specific convergence targets defined by BF16’s final metrics (perplexities of 3.107, 2.996, 3.095 and accuracy of 73.04%). For example, for BERT-large, “105%” means we measure the time it takes to reach the perplexity of $3.107 \cdot 1.05 \approx 3.22$, and for LLaMA 1B MMLU, “99%” means we measure the time it takes to reach the accuracy of $73.04 \cdot 0.99 \approx 72.3\%$. Bars are omitted for methods that do not reach the specified target.

5.1 Ring all-reduce

We next evaluate DYNAMIQ’s end-to-end performance with ring all-reduce, demonstrating that it offers substantial acceleration over BF16 and significantly better TTA curves than all other tested compression schemes. Importantly, DYNAMIQ outperforms MXFP8 across all workloads despite using a lower bit budget, which is attributed to its ability to maintain both low compression error and to incur small computational overhead despite its “involved” two-phase workflow.

Time to accuracy (TTA). As highlighted in Figure 4 and further detailed in Figure 5, DYNAMIQ consistently produces better TTA curves across all workloads. For instance, on Gemma 1B, DYNAMIQ achieves target perplexity 18%–28% faster than MXFP8 and MXFP6 respectively, while THC, Omnireduce, and MXFP4 either converge slower than the BF16 baseline or do not reach target accuracy due to excessive compression error. Similarly, for LLaMA fine-tuning, DYNAMIQ reaches 72.38% (99% of BF16’s accuracy) accuracy approximately 34.5% faster than MXFP8 and 40.8% faster than BF16. Crucially, DYNAMIQ maintains final accuracy within 0.1% of uncompressed BF16 across all scenarios, whereas other compression schemes exhibit degradation of up to 2.5%.

To shed more light on the performance of the different schemes, we next display their throughput and compression error measurements separately. Then, we show TTA curves for DYNAMIQ under different bit-budgets, clarifying our choice of five bits per-parameter.

Throughput. The TTA gains of DYNAMIQ are partially attributable to its improved training throughput, as shown in Figure 6. It is evident that the compression overhead of DYNAMIQ remains small, as gradient compression on GPUs is typically memory-bound rather than compute-bound [6]. Accordingly, by leveraging fused kernels, DYNAMIQ ensures a coalesced memory access pattern where each gradient coordinate is accessed only once, maintaining parity with the memory transaction volume of MXFP8 (see Table 2). In contrast, the randomized Hadamard transform used in THC [56] requires $O(\log d)$ additional global GPU memory accesses, creating a bottleneck that consumes up to 42% of the training round time in the LLaMA 1B workload.

Compression scheme	Global Memory Transactions
BF16	$4 + 4 \cdot AR$
DYNAMIQ	$22 + 11.875 \cdot AR$
MXFP8	$18 + 13 \cdot AR$
THC	$74 + 2 \cdot AR$

Table 2: Estimated extra DRAM memory transactions (bytes per coordinate) for different all-reduce compression schemes, excluding NIC-GPU data transfer. $AR = \frac{n-1}{n} \in [\frac{1}{2}, 1)$ denotes the per-worker data fraction transferred during reduce-scatter and all-gather phases.

Workload	BERT-large MaskedLM	LLaMA 1B Chat	Gemma 1B Chat	LLaMA 1B MMLU
DYNAMIQ	0.00217	0.00149	0.00122	0.00096
MXFP8	0.00591	0.00320	0.00308	0.00299
MXFP6	0.02332	0.01350	0.01458	0.01298
MXFP4	0.12080	0.11059	0.11583	0.09039
OR	0.15499	0.08044	0.04676	0.04530
THC	0.00897	0.11978	0.15168	0.19599

Table 3: Comparing compression error (vNMSE) for LLM training (fine-tuning) on an 8-GPU/4-worker testbed using ring all-reduce. In this table, we compute the average vNMSE over the entire end-to-end training process. The full version of round-to-vNMSE curves is shown in Figure 18.

Compression error. As detailed in Table 3, DYNAMIQ demonstrates better fidelity to uncompressed gradients, achieving 2.5–3× lower vNMSE than MXFP8 and orders-of-magnitude lower vNMSE than MXFP4, THC, and OmniReduce. These results further clarify the performance trade-offs observed earlier: while MXFP4 offers higher throughput (Figure 6), its excessive error slows-down convergence and degrades final accuracy. Similarly, OmniReduce underperforms in these benchmarks because it relies on gradient sparsity and skewness (i.e., a large fraction of near-zero entries), an attribute largely absent in dense LLM gradients [92].

DYNAMIQ’s bit-budget ablation. To justify our choice of using $b = 5$ bit-per-coordinate for DYNAMIQ, we next evaluate the impact of b on DYNAMIQ’s TTA and convergence accuracy. Figure 7 presents results for the LLaMA 1B MMLU workload, demonstrating that $b = 5$ indeed achieves the best trade-off in this scenario. Reducing

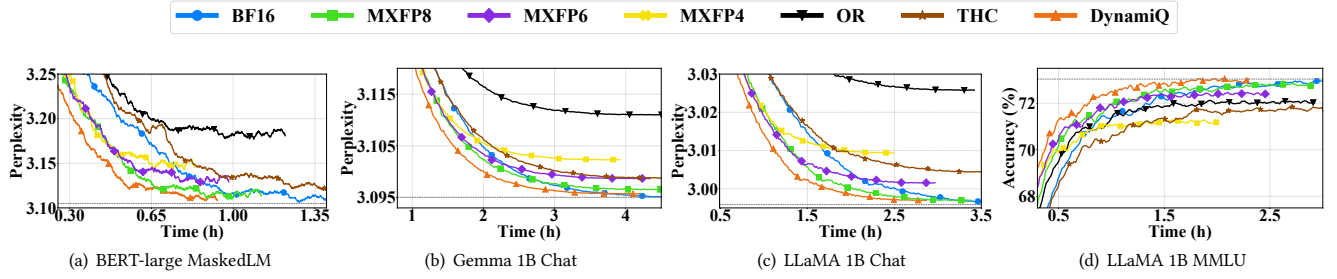


Figure 5: Zoomed-in Time to Accuracy (TTA) curves for LLM training (fine-tuning) on an 8-GPU/4-worker testbed using ring all-reduce. Horizontal dashed lines indicate the final BF16 accuracy. As mentioned, MXFP4 and MXFP6 curves represent a best-case scenario based on upper-bound throughput estimation. DYNAMIQ is the only method to consistently converge faster than BF16 while roughly matching its perplexity and accuracy, followed by MXFP8. Although alternatives like THC and OR also show faster-than-baseline initial convergence (see Appendix Fig. 14), their performance ultimately stalls due to high compression error (see Appendix Fig. 18).

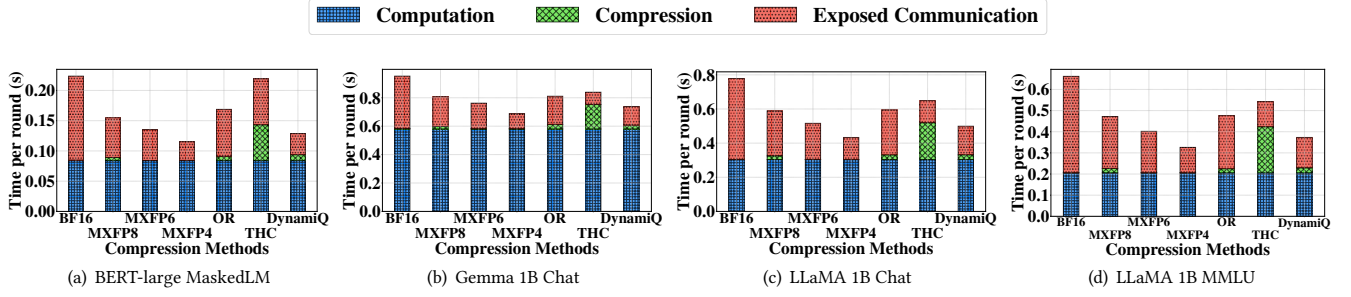


Figure 6: Breakdown of training time into computation, exposed communication, and compression overhead. Compression refers to the extra time-per-round latency introduced with all compression operations, including fused kernels and reordering costs, that cannot overlap with computation. Exposed communication refers to the portion of communication latency not overlapped with computation or compression thus directly contributes to training duration.

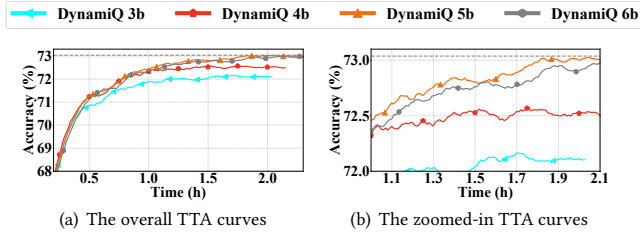


Figure 7: DynamiQ's bit-budget ablation. Displaying TTA curves for varying overall bit budget for training (fine-tuning) LLaMA 1B on the MMLU workload on an 8-GPU/4-worker testbed using ring all-reduce. The horizontal dashed lines represent the convergence accuracy of BF16.

the bit budget below this threshold increases compression error and degrades final accuracy, while increasing b yields no accuracy gains and, as detailed in Table 4, merely reduces throughput due to higher communication volumes.

5.2 Ring all-reduce over a shared network

In many setups, the training job may not run in isolation and has to share the network with other jobs or tenants. For example, multi-tenancy in cloud providers is a common practice for maximizing GPU utilization [25, 46]. Accordingly, in this experiment, we launch three additional DDP processes that continuously perform ring all-reduce operations, competing with the training job for bandwidth.

Method	LLaMA 1B MMLU		Gemma 1B Chat	
	vNMSE	Thp.	vNMSE	Thp.
DYNAMIQ 3b	0.01603	3.051	0.02334	1.440
DYNAMIQ 4b	0.00589	2.842	0.00831	1.397
DYNAMIQ 5b	0.00096	2.604	0.00122	1.353
DYNAMIQ 6b	0.00059	2.390	0.00053	1.306
MXFP8	0.00299	2.123	0.00308	1.246

Table 4: DynamiQ's bit-budget ablation. Displaying Throughput (in rounds per second) and vNMSE for varying overall bit budget for training (fine-tuning) LLaMA 1B on the MMLU workload on an 8-GPU/4-worker testbed using ring all-reduce. Results for MXFP8 are displayed for comparison.

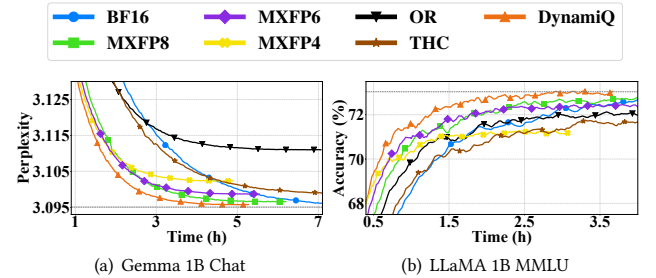


Figure 8: Zoomed-in TTA curves over a shared network. The full curves and additional results appear in Section D.

Method	Acc (%)	vNMSE
BF16	73.04	0
DYNAMIQ	73.04	0.00067
MXFP8	72.86	0.00203
MXFP6	72.46	0.02008
MXFP4	71.59	0.17058

Table 5: Evaluation with butterfly all-reduce on the LLaMA 1B MMLU workload. We list the final accuracy the model converges, to relative to the BF16 baseline, and average quantization error (vNMSE).

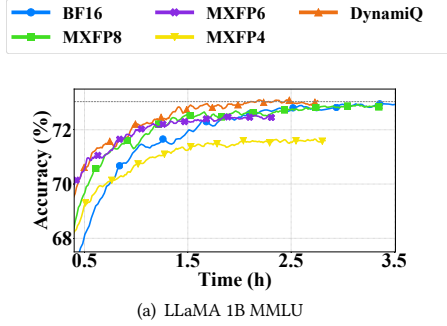


Figure 9: The zoomed-in TTA of DYNAMIQ with butterfly all-reduce compared with the baselines. The full version is shown in Figure 16 in the Appendix.

Figure 8 shows, as expected, that the compression methods', and particularly DYNAMIQ's, TTA advantage over the BF16 baseline increases under bandwidth contention. For example, on Gemma 1B + Chat, DYNAMIQ's advantage over MXFP8 increases from 16% in isolation to 21.5% over a shared network. Likewise, in the LLaMA 1B + MMLU workload, the advantage increases from 34.5% to 40.2%. Interestingly, the exposed communication time is shorter than 4× the time in isolation, as the different jobs converge to transmitting only on partially overlapping timeframes.

Finally, in the interest of space, full TTA curves and additional results are deferred to in Section D.

5.3 Butterfly all-reduce

We proceed with an experiment with butterfly all-reduce [78], which reduces the number of hops to logarithmic in the number of workers, thereby reducing latency. Interestingly, it also reduces quantization error, as fewer re-quantizations are needed and the summed partial sums on the aggregation path tend to have a closer order of magnitude [67].

Figure 9 depicts that, on the LLaMA 1B MMLU benchmark, DYNAMIQ achieves better TTA and, in particular, higher final accuracy than the MXFP4, MXFP6 and MXFP8 baselines. Specifically, DYNAMIQ attains an accuracy of 72.38% – corresponding to 99% of BF16's final accuracy—12.0% faster than MXFP8; this advantage further increases to 37.8% when targeting 99.5% of BF16's final accuracy. In addition, the microscaling baselines exhibit measurable degradation in final accuracy (Table 5), whereas DYNAMIQ achieves a final accuracy comparable to BF16. This is explained by DYNAMIQ's lower vNMSE (quantization error), as Table 5 shows.

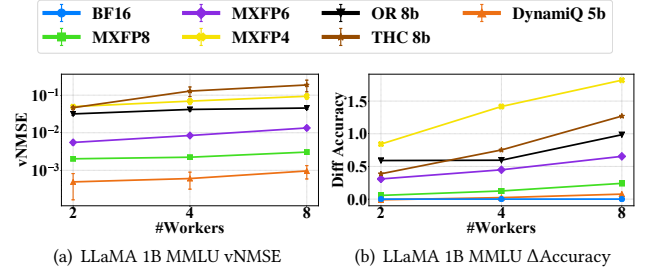


Figure 10: Scalability evaluation on the LLaMA + MMLU task with 2 to 8 workers, measuring the vNMSE and the MMLU's accuracy with respect to the BF16 baseline.

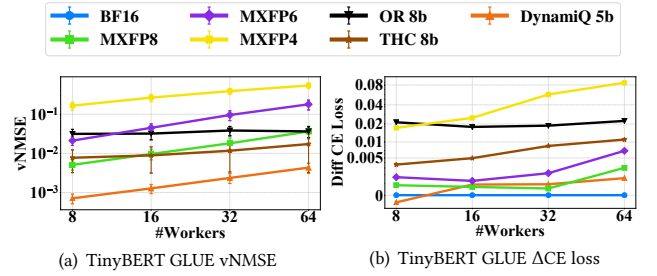


Figure 11: Scalability evaluation on the TinyBERT + GLUE task with 8 to 64 workers, measuring vNMSE and the cross entropy loss with respect to the BF16 baseline.

Lastly, we argue that this trend is expected to continue as the number of workers increases and provide theoretical intuition to support this in Section B.

6 Simulation Studies

We next test the scalability of DYNAMIQ and perform a parametric study to shed more light on its individual design components.

6.1 Scalability analysis

Setup and Methodology. We evaluate scalability by varying the worker count n from 2 to 64 across two workloads: LLaMA 1B MMLU (2–8 workers) and the smaller TinyBERT [48] on GLUE [83] (8–64 workers). In all experiments, we utilize ring all-reduce and benchmark performance against the BF16 baseline, measuring quantization error (vNMSE) and final accuracy (LLaMA 1B) and cross-entropy (CE) loss (TinyBERT). For THC, we adopt the authors' recommendation to allocate 12 bits for $n > 8$ to prevent gradient overflow during aggregation.

LLaMA 1B. As the worker count increases, both the vNMSE and the accuracy degradation naturally increase across all methods. However, as shown in Figure 10, DYNAMIQ shows better scaling properties compared to the baselines nearing the accuracy of BF16 even with 8 workers.

TinyBERT. Extending the analysis to larger clusters with TinyBERT, Figure 11 confirms that DYNAMIQ consistently achieves the lowest vNMSE among all compression schemes up to 64 workers. Consequently, it yields the final accuracy closest to the BF16 baseline (Figure 11(b)). We note that inherent training variance in small models leads to minor fluctuations. For example, DYNAMIQ slightly

Method	LLaMA 1B Chat	LLaMA 1B MMLU
Uniform quantization	0.1278	0.1207
Non-uniform quantization	0.0707	0.0664
+ Variable bitwidth allocation	0.0198	0.0130
+ Hierarchical quantization	0.0138	0.0092
+ Correlated rounding	0.0091	0.0059

Table 6: vNMSE comparisons among different variants of DYNAMIQ for the LLaMA 1B Chat and LLaMA 1B MMLU workloads. For hierarchical quantization, we use a group size of 16 instead of 32.

Number of bits $\bar{b}$	5	6	8.5
DynamiQ-ring	4.76	0.751	0.0105
MXFP8-ring	–	–	6.11
DynamiQ-butterfly	0.0336	3.23×10^{-3}	2.75×10^{-5}
MXFP8-butterfly	–	–	0.0353

Table 7: Compression error (vNMSE) in a simulation with synthetic data and a DP dimension of $n = 8,192$, and a total of 32768 super-groups of 256 coordinates per worker.

outperforms BF16 at $n = 8$ or MXFP8, showing marginally lower CE loss at $n = 16, 32$. However, the overall trend confirms that DYNAMIQ’s more scalable and stable than other compression methods.

Finally, we observe that, as illustrated in Figure 11(a), THC and OR display slower vNMSE growth as n scales. For THC, this results from increasing the allocation from 8 to 12 bits for $n > 8$ to prevent overflows (satisfying $b \geq \lceil \log(15n + 1) \rceil$). However, as a strategy, this remains effective only up to $n = 64$. For OmniReduce ($b = 8$), the error profile at this scale is determined by its sparsification policy, which consistently discards the bottom 50% of gradients.

6.2 Large-scale simulation

To evaluate DYNAMIQ at a large scale, we simulate compressed all-reduce with a DP dimension of $n = 8,192$. To model the spatial locality and scale skew observed in Section 2.2 and Fig. 1, we assign each super-group a magnitude parameter, which determines the variance of all coordinates in that super-group.

Synthetic data. We model the magnitude $M_{\mathcal{G}}$ of a super-group $\mathcal{G}$ as a mixture of two LogNormal distributions; this captures well the empirical distribution of Figure 1(c), as described in Section E. For each $\mathcal{G}$ (of size 256), we sample each of its coordinates, for each worker, i.i.d. from $\mathcal{N}(0, \frac{M_{\mathcal{G}}^2}{256})$. Consequently, coordinates within a super-group share a common scale, preserving spatial locality, while scales across super-groups span several orders of magnitude.

Setup. We evaluate communication budgets of $\bar{b} \in \{5, 6, 8.5\}$ bits per coordinate under both ring and butterfly all-reduce. We compare DYNAMIQ against MXFP8-e4m3 (we omit other baselines since they overflow in this setting, yielding a prohibitively large vNMSE). Because MXFP8-e4m3 has a fixed communication cost of $\bar{b} = 8.5$, we report it only at that budget. For DYNAMIQ, we allow bitwidths from $W = \{2, 4, 8, 16\}$.

Results. As shown in Table 7, DYNAMIQ achieves substantially lower vNMSE than MXFP8. With ring all-reduce and $\bar{b} = 8.5$, DYNAMIQ achieves a vNMSE of 0.0105, which is ≈ 582 times lower than MXFP8. On the other hand, DYNAMIQ’s vNMSE with 5 bits roughly matches with MXFP8 that consumes $\bar{b} = 8.5$ bits. Similarly,

with butterfly all-reduce and $\bar{b} = 8.5$, DYNAMIQ achieves a vNMSE of 2.75×10^{-5} , which is 1283 times lower than MXFP8. We note that at this large DP dimension, MXFP8-ring and DYNAMIQ-ring with $\bar{b} \in \{5, 6\}$ incur unacceptably large vNMSE. Thus, for DYNAMIQ-ring with this DP dimension, we recommend using $\bar{b} = 8.5$, where for DYNAMIQ-butterfly, $\bar{b} = 6$ suffices.

We partially attribute these gains to two features of DYNAMIQ. First, its bit-allocation scheme assigns up to 16 bits to rare, high-norm super-groups without imposing this cost on all coordinates. Second, its decompress–accumulate–recompress procedure updates the group scales as the partial sums evolve, allowing the quantization range to track the data well throughout the all-reduce. In contrast, MXFP8-e4m3 reserves four of its eight per-value bits for the exponent to represent a wide dynamic range uniformly.

6.3 Parametric study

We next isolate the impact of DYNAMIQ’s optimization components, namely, variable bitwidth allocation, non-uniform quantization, hierarchical quantization, and correlated rounding, on compression error. The group size is set to 32 and reduced to 16 when hierarchical quantization (with INT8 scaling parameters) is used.

Table 6 demonstrates that the cumulative application of these techniques reduces vNMSE by a factor of 14× for LLaMA 1B Chat and 22× for MMLU. Variable bitwidth allocation serves as the primary driver, improving quantization accuracy by 3.5–5.1×. Complementary techniques provide significant additive gains: non-uniform quantization reduces vNMSE by $\sim 45\%$ (see Appendix Figure 12), hierarchical quantization by $\sim 30\%$, and correlated rounding by $\sim 35\%$. As discussed in Section 5.1, this order-of-magnitude reduction in error is essential for maintaining model accuracy comparable to uncompressed baselines. Crucially, as was shown in Figure 6, these enhancements introduce only a small computational overhead.

7 Related Work

Gradient Compression and the Shift to all-reduce. Gradient compression is a well-established strategy for accelerating distributed data-parallel (DDP) training by mitigating communication bottlenecks [55, 71, 72, 90]. While many such methods have been proposed [16, 17, 24, 28, 36, 50, 54, 56, 60, 74, 82, 84, 86, 87], these were designed for the parameter server architecture [47, 55]. Indeed, the recent paradigm shift toward multi-hop all-reduce for scaling LLM training [1, 10, 35, 45, 72] reveals significant limitations in these approaches. For instance, sparsity-based methods like OmniReduce [36] struggle to merge local TopK chunks efficiently across decentralized topologies. Similarly, quantization schemes such as QSGD [16], THC [56], and Terngrad [90] are prone to gradient overflow during the aggregation of partial sums, a fundamental issue in multi-hop topologies that worsens with system size [42]. While microscaling-based methods (e.g., MXFP4) [7, 68] alleviate this, they do not eliminate overflow/underflow entirely. By contrast, DYNAMIQ is explicitly architected for multi-hop all-reduce, employing hop-wise decompression/recompression to strictly prevent overflow and utilizing variable bitwidth allocation to ensure robustness. **Compression Error and Scalability.** Although LLMs exhibit some tolerance for compression noise, excessive error significantly degrades convergence stability and final accuracy [52, 53]. Existing

schemes often prioritize inference hardware compatibility or sparsity over minimizing the error (vNMSE). For example, microscaling techniques [70] optimize for GPU throughput but lack advanced error-reduction mechanisms, while OmniReduce relies on gradient sparsity that is largely absent in dense LLM updates. Furthermore, maintaining bounded error as the worker count n increases presents a significant challenge; errors accumulate hop-by-hop, typically necessitating a bitwidth growth that is proportional to the logarithm of the aggregation path length (as in THC) to prevent overflow. We empirically observe that for DYNAMIQ this growth is slower, but leave further investigation for future work.

Hardware-Aware Implementation. Gradient compression on GPUs is predominantly memory-bound rather than compute-bound; performance is dictated by HBM bandwidth rather than floating-point throughput [3, 5, 6, 40]. Consequently, efficient implementations must minimize HBM transactions, ideally ensuring sequential, single-pass access via kernel fusion [12]. Methods that fail to respect this constraint incur substantial overhead. A notable example is THC, where the Hadamard transform [43] requires $O(\log d)$ passes over memory, creating a bottleneck. DYNAMIQ avoids such overhead by leveraging fused kernels that keep intermediate results in registers or shared memory, maintaining a memory access pattern comparable to standard uncompressed updates.

Mixed-precision training. An emerging technique to accelerate training is using lower precision arithmetics [13, 26, 27, 66, 79, 85, 91, 93]. This is motivated by new hardware capabilities that deliver higher throughput for low-precision operations [14]. Current best practices keep certain fields (e.g., outlier values or accumulators) in higher precision while using low precision elsewhere [85]. Recently, researchers proposed running the entire training process in low precision [26, 79], often leading to a degradation of accuracy that can be acceptable in certain scenarios.

Sharded models. When models are too large to fit on a GPU, practitioners shard them across multiple workers, each holding a portion [69, 94]. In such cases, one may not need an all-reduce operation but rather only the reduce-scatter phase, since gradients and weights are split across GPUs. DYNAMIQ can seamlessly integrate with this approach by decompressing at the end of the reduce-scatter phase.

8 Discussion and Limitations

Implications for large-scale pre-training. Our testbed evaluation focuses on fine-tuning due to limited hardware resources, but it already covers DP dimensions up to 64. In real pre-training clusters, this can correspond to much larger total GPU counts, since 3D parallelism keeps the DP dimension substantially smaller than the total number of GPUs; for example, even clusters with up to 100K GPUs may still operate with comparable DP dimensions [29, 58, 62]. Thus, our testbed results are directly relevant to realistic large-scale pre-training deployments. For even larger deployments, such as future million-GPU clusters, scaling DYNAMIQ primarily requires either increasing the bit budget or using hierarchical aggregation. Our simulations show that DYNAMIQ remains accurate at DP dimension 8192 with an 8.5-bit budget, suggesting that modestly larger budgets can support much larger DP groups. Alternatively, hierarchical butterfly- or hypercube-style aggregation, which is

commonly needed at such scales to bound tail latency, can also reduce the effective aggregation depth.

Preserving model quality and numerical stability. For large-scale pre-training tasks, practitioners are often conservative about low-precision compression schemes [37, 38], because compression error can manifest as hidden instability that harms final model quality or convergence. For example, occasionally quantizing extreme values into an unsafe low-precision representation can lead to NaNs, causing model divergence, or otherwise substantially degrade accuracy. DYNAMIQ’s super-group bitwidth allocation scheme (Section 3.2) naturally supports preserving such outlier values in the training precision, e.g., BF16.

Compatibility with FSDP and MoE [73] gradients. DYNAMIQ is compatible with FSDP-style synchronization as FSDP decomposes synchronization into reduce-scatter and all-gather phases, to which DYNAMIQ’s multi-hop communication primitive can be applied as described in Section 3.4. For MoE gradients, where experts can be active or inactive, DYNAMIQ’s bit allocation can assign 0 bits to inactive expert super-groups while allocating more bits to active ones.

Implications for high-end training hardware. Our hardware evaluation uses A6000 GPUs with 768 GB/s GPU memory bandwidth and 100 Gb/s inter-node bandwidth. Modern clusters with, e.g., GB200-class GPUs provide faster scale-out networks, such as 400 Gb/s or 800 Gb/s links [63, 64], but also 16× higher FP16/BF16 FLOPS and 10.4× memory bandwidth [65]. Therefore, communication remains a bottleneck, especially under large data-parallel dimensions or shared-cluster contention.

9 Conclusion

In this paper, we presented DYNAMIQ, a practical gradient compression framework optimized for multi-hop all-reduce that can adjust to different bandwidth constraints and presents an attractive trade-off between communication overhead and accuracy. In contrast with existing gradient compression systems, which are designed for the parameter-server architecture and incur accuracy degradation when deployed to multi-hop all-reduce, DYNAMIQ preserves low compression error along the aggregation paths, resulting in accelerated training without compromising model accuracy.

We implement DYNAMIQ and evaluate its performance across diverse LLM training workloads using both ring and butterfly all-reduce. Our results show that DYNAMIQ consistently achieves significantly better time-to-accuracy compared to alternatives. Notably, DYNAMIQ reaches 99.9% of BF16 baseline accuracy with only 5 bits per coordinate, outperforming the state-of-the-art MXFP8. It is the *only* evaluated method to consistently maintain this fidelity while providing significant acceleration, a result driven by its fast, co-designed fused CUDA kernels. Our code is available at [41].

Acknowledgments

We thank the anonymous reviewers and our shepherd for their insightful feedbacks. Michael Mitzenmacher was supported in part by grants NSF CNS-2107078 and NSF DMS-2023528. Ran Ben Basat was supported in part by the Google Research Scholar Award.

References

- [1] 2017. Ring all reduce. <https://github.com/baidu-research/baidu-allreduce>. (2017).
- [2] 2019. BFloat16: The secret to high performance on Cloud TPUs. <https://cloud.google.com/blog/products/ai-machine-learning/bfloat16-the-secret-to-high-performance-on-cloud-tpus>. (2019).
- [3] 2020. Accelerating HPC Applications with NVIDIA Nsight Compute Roofline Analysis. <https://developer.nvidia.com/blog/accelerating-hpc-applications-with-nsight-compute-roofline-analysis>. (2020).
- [4] 2020. JAX: High performance array computing. <https://docs.jax.dev/en/latest/index.html>. (2020).
- [5] 2023. Mastering LLM Techniques: Inference Optimization. <https://developer.nvidia.com/blog/mastering-llm-techniques-inference-optimization/>. (2023).
- [6] 2023. Memory-Limited Layers User's Guide. <https://docs.nvidia.com/deeplearning/performance/dl-performance-memory-limited/index.html>. (2023).
- [7] 2023. OCP Microscaling Formats (MX) Specification. <https://www.opencompute.org/documents/ocp-microscaling-formats-mx-v1-0-spec-final-pdf>. (2023).
- [8] 2024. Collective operations in NCCL. <https://docs.nvidia.com/deeplearning/nccl/user-guide/docs/usage/collectives.html>. (2024).
- [9] 2024. GGUF format. <https://github.com/ggml-org/ggml/blob/master/docs/gguf.md>. (2024).
- [10] 2024. How Meta trains large language models at scale. <https://engineering.fb.com/2024/06/12/data-infrastructure/training-large-language-models-at-scale-meta/>. (2024).
- [11] 2024. NVIDIA Collective Communications Library (NCCL). <https://developer.nvidia.com/nccl>. (2024).
- [12] 2025. Advanced NVIDIA CUDA Kernel Optimization Techniques: Handwritten PTX. <https://developer.nvidia.com/blog/advanced-nvidia-cuda-kernel-optimization-techniques-handwritten-ptx/>. (2025).
- [13] 2025. NVFP4 Trains with Precision of 16-Bit and Speed and Efficiency of 4-Bit. <https://developer.nvidia.com/blog/nvfp4-trains-with-precision-of-16-bit-and-speed-and-efficiency-of-4-bit/>. (2025).
- [14] 2025. NVIDIA Blackwell Architecture Technical Brief. (2025). <https://resources.nvidia.com/en-us-blackwell-architecture>
- [15] Saurabh Agarwal, Hongyi Wang, Shivaram Venkataraman, and Dimitris Papailiopoulos. 2022. On the utility of gradient compression in distributed training systems. *Proceedings of Machine Learning and Systems* 4 (2022), 652–672.
- [16] Dan Alistarh, Demjan Grubic, Jerry Li, Ryota Tomioka, and Milan Vojnovic. 2017. QSGD: Communication-efficient SGD via gradient quantization and encoding. *Advances in neural information processing systems* 30 (2017).
- [17] Youhui Bai, Cheng Li, Quan Zhou, Jun Yi, Ping Gong, Feng Yan, Ruichuan Chen, and Yinlong Xu. 2021. Gradient compression supercharged high-performance data parallel dnn training. In *Proceedings of the ACM SIGOPS 28th Symposium on Operating Systems Principles*. 359–375.
- [18] Ran Ben-Basat, Yaniv Ben-Itzhak, Gal Mendelson, Michael Mitzenmacher, Amit Portnoy, and Shay Vargaftik. 2026. A note on turboquant and the earlier drive/eden line of work. *arXiv preprint arXiv:2604.18555* (2026).
- [19] Ran Ben-Basat, Yaniv Ben-Itzhak, Michael Mitzenmacher, and Shay Vargaftik. 2024. Optimal and approximate adaptive stochastic quantization. *Advances in Neural Information Processing Systems* 37 (2024), 94265–94291.
- [20] Ran Ben Basat, Yaniv Ben-Itzhak, Michael Mitzenmacher, and Shay Vargaftik. 2025. Better than Optimal: Improving Adaptive Stochastic Quantization Using Shared Randomness. *Proceedings of the ACM on Measurement and Analysis of Computing Systems* 9, 3 (2025), 1–44.
- [21] Ran Ben-Basat, William Kuszmaul, Michael Mitzenmacher, Amit Portnoy, and Shay Vargaftik. 2026. Quantizing With Randomized Hadamard Transforms: Efficient Heuristic Now Proven. *arXiv preprint arXiv:2605.06014* (2026).
- [22] Ran Ben Basat, Michael Mitzenmacher, and Shay Vargaftik. 2021. How to Send a Real Number Using a Single Bit (And Some Shared Randomness). In *48th International Colloquium on Automata, Languages, and Programming (ICALP 2021)*.
- [23] Ran Ben-Basat, Amit Portnoy, Gil Einziger, Yaniv Ben-Itzhak, and Michael Mitzenmacher. 2024. Accelerating Federated Learning with Quick Distributed Mean Estimation. In *ICML*.
- [24] Jeremy Bernstein, Yu-Xiang Wang, Kamyar Azizzadenesheli, and Animashree Anandkumar. 2018. signSGD: Compressed optimisation for non-convex problems. In *International Conference on Machine Learning*. PMLR, 560–569.
- [25] Jiamin Cao, Yu Guan, Kun Qian, Jiaqi Gao, Wencong Xiao, Jianbo Dong, Binzhang Fu, Dennis Cai, and Ennan Zhai. 2024. Crux: Gpu-efficient communication scheduling for deep learning training. In *Proceedings of the ACM SIGCOMM 2024 Conference*. 1–15.
- [26] Roberto L Castro, Andrei Panferov, Soroush Tabesh, Oliver Sieberling, Jiale Chen, Mahdi Nikdan, Saleh Ashkboos, and Dan Alistarh. 2025. Quartet: Native FP4 Training Can Be Optimal for Large Language Models. *arXiv preprint arXiv:2505.14669* (2025).
- [27] Mengzhao Chen, Wenqi Shao, Peng Xu, Jiahao Wang, Peng Gao, Kaipeng Zhang, Yu Qiao, and Ping Luo. 2024. EfficientQAT: Efficient Quantization-Aware Training for Large Language Models. *arXiv preprint arXiv:2407.11062* (2024).
- [28] Xiaoqi Chen, Shay Vargaftik, and Ran Ben-Basat. 2024. When ML Training Cuts Through Congestion: Just-in-Time Gradient Compression via Packet Trimming. In *Hotnets*.
- [29] Weiwei Chu, Xinfeng Xie, Jiecao Yu, Jie Wang, Amar Phanishayee, Chunqiang Tang, Yuchen Hao, Jianyu Huang, Mustafa Ozdal, Jun Wang, Vedanuj Goswami, Naman Goyal, Abhishek Kadian, Andrew Gu, Chris Cai, Feng Tian, Xiaodong Wang, Min Si, Pavan Balaji, Ching-Hsiang Chu, and Jongsoo Park. 2025. Scaling Llama 3 Training with Efficient Parallelism Strategies. In *Proceedings of the 52nd Annual International Symposium on Computer Architecture (ISCA '25)*. Association for Computing Machinery. <https://doi.org/10.1145/3695053.3731410>
- [30] Jeffrey Dean, Greg Corrado, Rajat Monga, Kai Chen, Matthieu Devin, Mark Mao, Marc' aurelio Ranzato, Andrew Senior, Paul Tucker, Ke Yang, Quoc Le, and Andrew Ng. 2012. Large Scale Distributed Deep Networks. In *Advances in Neural Information Processing Systems*, F. Pereira, C.J. Burges, L. Bottou, and K.Q. Weinberger (Eds.), Vol. 25. Curran Associates, Inc. https://proceedings.neurips.cc/paper_files/paper/2012/file/6aca97005c68f1206823815f66102863-Paper.pdf
- [31] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2018. Bert: Pre-training of deep bidirectional transformers for language understanding. *arXiv preprint arXiv:1810.04805* (2018).
- [32] Ning Ding, Yulin Chen, Bokai Xu, Yujia Qin, Zhi Zheng, Shengding Hu, Zhiyuan Liu, Maosong Sun, and Bowen Zhou. 2023. Enhancing Chat Language Models by Scaling High-quality Instructional Conversations. *arXiv preprint arXiv:2305.14233* (2023).
- [33] Ron Dorfman, Shay Vargaftik, Yaniv Ben-Itzhak, and Kfir Yehuda Levy. 2023. DoCoFL: Downlink compression for cross-device federated learning. In *International Conference on Machine Learning*. PMLR, 8356–8388.
- [34] Gil Einziger, Benny Fellman, Roy Friedman, and Yaron Kassner. 2018. Ice buckets: Improved counter estimation for network measurement. *IEEE/ACM Transactions on Networking* 26, 3 (2018), 1165–1178.
- [35] Grattafiori et al. 2024. The Llama 3 Herd of Models. (2024). [arXiv:cs.AI/2407.21783](https://arxiv.org/abs/2407.21783)
- [36] Jiawei Fei, Chen-Yu Ho, Atal N Sahu, Marco Canini, and Amedeo Sapio. 2021. Efficient sparse collective communication and its application to accelerate distributed deep learning. In *Proceedings of the 2021 ACM SIGCOMM 2021 Conference*. 676–691.
- [37] Maxim Fishman, Brian Chmiel, Ron Banner, and Daniel Soudry. 2025. Scaling FP8 Training to Trillion-Token LLMs. In *International Conference on Learning Representations*.
- [38] Kazuki Fujii, Taishi Nakamura, and Rio Yokota. 2024. Balancing Speed and Stability: The Trade-offs of FP8 vs. BF16 Training in LLMs. (2024). [arXiv:cs.LG/2411.08719](https://arxiv.org/abs/2411.08719)
- [39] Alexandru M. Gherghescu, Vlad-Andrei Bădoiu, Alexandru Agache, Mihai-Valentin Dumitru, Iuliu Vasilescu, Radu Mantu, and Costin Raiciu. 2024. I've Got 99 Problems But FLOPS Ain't One. In *Proceedings of the 23rd ACM Workshop on Hot Topics in Networks (HotNets '24)*. Association for Computing Machinery, New York, NY, USA, 195–204. <https://doi.org/10.1145/3696348.3696893>
- [40] Amir Gholami, Zhewei Yao, Sehoon Kim, Coleman Hooper, Michael W Mahoney, and Kurt Keutzer. 2024. Ai and memory wall. *IEEE Micro* 44, 3 (2024), 33–39.
- [41] Wenchen Han. 2026. Github code repository of DynamiQ artifact. <https://github.com/CharlesHan24/DynamiQ>. (2026).
- [42] Wenchen Han, Shay Vargaftik, Michael Mitzenmacher, Brad Karp, and Ran Ben Basat. 2024. Beyond Throughput and Compression Ratios: Towards High End-to-end Utility of Gradient Compression. In *Proceedings of the 23rd ACM Workshop on Hot Topics in Networks (HotNets '24)*. Association for Computing Machinery, New York, NY, USA, 186–194. <https://doi.org/10.1145/3696348.3696857>
- [43] A Hedayat and Walter Dennis Wallis. 1978. Hadamard matrices and their applications. *The annals of statistics* (1978), 1184–1238.
- [44] Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. 2020. Measuring massive multitask language understanding. *arXiv preprint arXiv:2009.03300* (2020).
- [45] Torsten Hoeffler, Tommaso Bonato, Daniele De Sensi, Salvatore Di Girolamo, Shigang Li, Marco Heddes, Jon Belk, Deepak Goel, Miguel Castro, and Steve Scott. 2022. HammingMesh: a network topology for large-scale deep learning. In *Proceedings of the International Conference on High Performance Computing, Networking, Storage and Analysis (SC '22)*. IEEE Press, Article 11, 18 pages.
- [46] Changho Hwang, Taehyun Kim, Sunghyun Kim, Jinwoo Shin, and Kyoungsoo Park. 2021. Elastic resource sharing for distributed deep learning. In *18th USENIX Symposium on Networked Systems Design and Implementation (NSDI 21)*. 721–739.
- [47] Yimin Jiang, Yibo Zhu, Chang Lan, Bairen Yi, Yong Cui, and Chuanxiong Guo. 2020. A unified architecture for accelerating distributed {DNN} training in heterogeneous {GPU/CPU} clusters. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*. 463–479.
- [48] Xiaoqi Jiao, Yichun Yin, Lifeng Shang, Xin Jiang, Xiao Chen, Linlin Li, Fang Wang, and Qun Liu. 2019. Tinybert: Distilling bert for natural language understanding. *arXiv preprint arXiv:1909.10351* (2019).
- [49] Sai Praneeth Karimireddy, Quentin Rebjock, Sebastian Stich, and Martin Jaggi. 2019. Error feedback fixes signsgd and other gradient compression schemes. In

- International Conference on Machine Learning*. PMLR, 3252–3261.
- [50] Soojeong Kim, Gyeong-In Yu, Hoin Park, Sungwoo Cho, Eunji Jeong, Hyeonmin Ha, Sanha Lee, Joo Seong Jeong, and Byung-Gon Chun. 2019. Parallax: Sparsity-aware data parallel training of deep neural networks. In *Proceedings of the Fourteenth EuroSys Conference 2019*. 1–15.
 - [51] Diederik Kinga, Jimmy Ba Adam, et al. 2015. A method for stochastic optimization. In *International conference on learning representations (ICLR)*, Vol. 5. California.
 - [52] Joonhyung Lee, Jeongin Bae, Byeongwook Kim, Se Jung Kwon, and Dongsoo Lee. 2024. To FP8 and Back Again: Quantifying Reduced Precision Effects on LLM Training Stability. *arXiv preprint arXiv:2405.18710* (2024).
 - [53] Wonyeol Lee, Rahul Sharma, and Alex Aiken. 2023. Training with mixed-precision floating-point assignments. *arXiv preprint arXiv:2301.13464* (2023).
 - [54] Haoyu Li, Yuchen Xu, Jiayi Chen, Rohit Dwivedula, Wenfei Wu, Keqiang He, Aditya Akella, and Dachyeok Kim. 2024. Accelerating Distributed Deep Learning using Lossless Homomorphic Compression. *arXiv preprint arXiv:2402.07529* (2024).
 - [55] Mu Li, David G Andersen, Jun Woo Park, Alexander J Smola, Amr Ahmed, Vanja Josifovski, James Long, Eugene J Shekita, and Bor-Yiing Su. 2014. Scaling distributed machine learning with the parameter server. In *11th USENIX Symposium on operating systems design and implementation (OSDI 14)*. 583–598.
 - [56] Minghao Li, Ran Ben Basat, Shay Vargaftik, ChonLam Lao, Kevin Xu, Michael Mitzenmacher, and Minlan Yu. 2024. {THC}: Accelerating Distributed Deep Learning Using Tensor Homomorphic Compression. In *21st USENIX Symposium on Networked Systems Design and Implementation (NSDI 24)*. 1191–1211.
 - [57] Shen Li, Yanli Zhao, Rohan Varma, Omkar Salpekar, Pieter Noordhuis, Teng Li, Adam Paszke, Jeff Smith, Brian Vaughan, Pritam Damania, et al. 2020. Pytorch distributed: Experiences on accelerating data parallel training. *arXiv preprint arXiv:2006.15704* (2020).
 - [58] Llama Team, AI @ Meta. 2024. The Llama 3 Herd of Models. *arXiv preprint arXiv:2407.21783* (2024). <https://doi.org/10.48550/arXiv.2407.21783>
 - [59] Ilya Loshchilov and Frank Hutter. 2017. Decoupled weight decay regularization. *arXiv preprint arXiv:1711.05101* (2017).
 - [60] Ahmed M Abdelmoniem, Ahmed Elzanaty, Mohamed-Slim Alouini, and Marco Canini. 2021. An efficient statistical-based gradient compression technique for distributed training systems. *Proceedings of Machine Learning and Systems 3* (2021), 297–322.
 - [61] Stephen Merity, Caiming Xiong, James Bradbury, and Richard Socher. 2016. Pointer sentinel mixture models. *arXiv preprint arXiv:1609.07843* (2016).
 - [62] Deepak Narayanan, Mohammad Shoyebi, Jared Casper, Patrick LeGresley, Mostofa Patwary, Vijay Korthikanti, Dmitri Vainbrand, Prethvi Kashinkunti, Julie Bernauer, Bryan Catanzaro, Amar Phanishayee, and Matei Zaharia. 2021. Efficient Large-Scale Language Model Training on GPU Clusters Using Megatron-LM. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC '21)*. Association for Computing Machinery. <https://doi.org/10.1145/3458817.3476209>
 - [63] NVIDIA. 2026. NVIDIA ConnectX-7 400G Adapters: Datasheet. NVIDIA Datasheet. (2026). <https://www.nvidia.com/content/dam/en-zz/Solutions/networking/infiniband/connectx-7-datasheet.pdf> Accessed: 2026-06-17.
 - [64] NVIDIA. 2026. NVIDIA ConnectX-7 Adapter Cards User Manual. NVIDIA Networking Documentation. (2026). <https://networking-docs.nvidia.com/connectx7hw> Accessed: 2026-06-17.
 - [65] NVIDIA. 2026. NVIDIA GB200 NVL72. NVIDIA Data Center Product Page. (2026). <https://www.nvidia.com/en-us/data-center/gb200-nvl72/> Accessed: 2026-06-17.
 - [66] Andrei Panferov, Jiale Chen, Soroush Tabesh, Roberto L Castro, Mahdi Nikdan, and Dan Alistarh. [n. d.]. QuEST: Training Accurate LLMs over Highly-Compressed Weights and Activation. In *Sparsity in LLMs (SLLM): Deep Dive into Mixture of Experts, Quantization, Hardware, and Inference*.
 - [67] Pitch Patarasuk and Xin Yuan. 2009. Bandwidth optimal all-reduce algorithms for clusters of workstations. *J. Parallel and Distrib. Comput.* 69, 2 (2009), 117–124.
 - [68] Houwen Peng, Kan Wu, Yixuan Wei, Guoshuai Zhao, Yuxiang Yang, Ze Liu, Yifan Xiong, Ziyue Yang, Bolin Ni, Jingcheng Hu, et al. 2023. Fp8-lm: Training fp8 large language models. *arXiv preprint arXiv:2310.18313* (2023).
 - [69] Samyam Rajbhandari, Jeff Rasley, Olatunji Ruwase, and Yuxiong He. 2020. Zero: Memory optimizations toward training trillion parameter models. In *SC20: International Conference for High Performance Computing, Networking, Storage and Analysis*. IEEE, 1–16.
 - [70] Bitu Darvish Rouhani, Ritchie Zhao, Ankit More, Mathew Hall, Alireza Khodamoradi, Summer Deng, Dhruv Choudhary, Marius Cornea, Eric Dellinger, Kristof Denolf, et al. 2023. Microscaling data formats for deep learning. *arXiv preprint arXiv:2310.10537* (2023).
 - [71] Amedeo Sapia, Marco Canini, Chen-Yu Ho, Jacob Nelson, Panos Kalnis, Changhoon Kim, Arvind Krishnamurthy, Masoud Moshref, Dan Ports, and Peter Richtárik. 2021. Scaling distributed machine learning with {In-Network} aggregation. In *18th USENIX Symposium on Networked Systems Design and Implementation (NSDI 21)*. 785–808.
 - [72] Daniele De Sensi, Tommaso Bonato, David Saam, and Torsten Hoefler. 2024. Swing: Short-cutting Rings for Higher Bandwidth Allreduce. In *21st USENIX Symposium on Networked Systems Design and Implementation (NSDI 24)*. USENIX Association, Santa Clara, CA, 1445–1462. <https://www.usenix.org/conference/nsdi24/presentation/de-sensi>
 - [73] Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarczyk, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. 2017. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. *arXiv preprint arXiv:1701.06538* (2017).
 - [74] Sebastian U Stich, Jean-Baptiste Cordonnier, and Martin Jaggi. 2018. Sparsified SGD with memory. *Advances in neural information processing systems* 31 (2018).
 - [75] Ananda Theertha Suresh, Ziteng Sun, Jae Ro, and Felix Yu. 2022. Correlated quantization for distributed mean estimation and optimization. In *International Conference on Machine Learning*. PMLR, 20856–20876.
 - [76] Zhenheng Tang, Zichen Tang, Junlin Huang, Xinglin Pan, Rudan Yan, Yuxin Wang, Amelie Chi Zhou, Shaohuai Shi, Xiaowen Chu, and Bo Li. 2025. DreamDDP: Accelerating Data Parallel Distributed LLM Training with Layer-wise Scheduled Partial Synchronization. (2025). [arXiv:cs.DC/2502.11058](https://arxiv.org/abs/2502.11058) <https://arxiv.org/abs/2502.11058>
 - [77] Gemma Team, Aishwarya Kamath, Johan Ferret, Shreya Pathak, Nino Vieillard, Ramona Merhej, Sarah Perrin, Tatiana Matejovicova, Alexandre Ramé, Morgane Rivière, et al. 2025. Gemma 3 technical report. *arXiv preprint arXiv:2503.19786* (2025).
 - [78] Rajeev Thakur, Rolf Rabenseifner, and William Gropp. 2005. Optimization of collective communication operations in MPICH. *The International Journal of High Performance Computing Applications* 19, 1 (2005), 49–66.
 - [79] Albert Tseng, Tao Yu, and Youngsuk Park. 2025. Training LLMs with MXFP4. In *Proceedings of The 28th International Conference on Artificial Intelligence and Statistics*. 1630–1638.
 - [80] Shay Vargaftik, Ran Ben Basat, Amit Portnoy, Gal Mendelson, Yaniv Ben Itzhak, and Michael Mitzenmacher. 2022. Eden: Communication-efficient and robust distributed mean estimation for federated learning. In *International Conference on Machine Learning*. PMLR, 21984–22014.
 - [81] Shay Vargaftik, Ran Ben-Basat, Amit Portnoy, Gal Mendelson, Yaniv Ben-Itzhak, and Michael Mitzenmacher. 2021. Drive: One-bit distributed mean estimation. *Advances in Neural Information Processing Systems* 34 (2021), 362–377.
 - [82] Thijs Vogels, Sai Praneeth Karimireddy, and Martin Jaggi. 2019. PowerSGD: Practical low-rank gradient compression for distributed optimization. *Advances in Neural Information Processing Systems* 32 (2019).
 - [83] Alex Wang, Amanpreet Singh, Julian Michael, Felix Hill, Omer Levy, and Samuel R Bowman. 2018. GLUE: A multi-task benchmark and analysis platform for natural language understanding. *arXiv preprint arXiv:1804.07461* (2018).
 - [84] Hongyi Wang, Scott Sievert, Shengchao Liu, Zachary Charles, Dimitris Papailiopoulos, and Stephen Wright. 2018. Atomo: Communication-efficient learning via atomic sparsification. *Advances in neural information processing systems* 31 (2018).
 - [85] Ruizhe Wang, Yeyun Gong, Xiao Liu, Guoshuai Zhao, Ziyue Yang, Baining Guo, Zhengjun Zha, and Peng Cheng. 2025. Optimizing large language model training using fp4 quantization. In *Forty-second International Conference on Machine Learning*.
 - [86] Zhuang Wang, Haibin Lin, Yibo Zhu, and TS Eugene Ng. 2023. Hi-Speed DNN Training with Espresso: Unleashing the Full Potential of Gradient Compression with Near-Optimal Usage Strategies. In *Proceedings of the Eighteenth European Conference on Computer Systems*. 867–882.
 - [87] Zhuang Wang, Xinyu Crystal Wu, Zhaozhuo Xu, and TS Eugene Ng. 2023. CUP-CAKE: A Compression Optimizer for Scalable Communication-efficient Distributed Training. In *Proceedings of the Sixth Conference on Machine Learning and Systems (MLSys' 23)*. Proceedings of the Sixth Conference on Machine Learning and Systems (MLSys' 23).
 - [88] Ertza Warraich, Ali Imran, Annus Zulfiqar, Shay Vargaftik, Sonia Fahmy, and Muhammad Shahbaz. 2025. OptiNIC: A Resilient and Tail-Optimal RDMA NIC for Distributed ML Workloads. *arXiv preprint arXiv:2512.22743* (2025).
 - [89] Ertza Warraich, Omar Shabtai, Khalid Manaa, Shay Vargaftik, Yonatan Piasetzky, Matty Kadosh, Lalith Suresh, and Muhammad Shahbaz. 2025. {OptiReduce}: Resilient and {Tail-Optimal} {AllReduce} for Distributed Deep Learning in the Cloud. In *22nd USENIX Symposium on Networked Systems Design and Implementation (NSDI 25)*. 685–703.
 - [90] Wei Wen, Cong Xu, Feng Yan, Chunpeng Wu, Yandan Wang, Yiran Chen, and Hai Li. 2017. Terngrad: Ternary gradients to reduce communication in distributed deep learning. *Advances in neural information processing systems* 30 (2017).
 - [91] Zeyu Zhang, Haiying Shen, Shay Vargaftik, Ran Ben Basat, Michael Mitzenmacher, and Minlan Yu. 2025. Hack: Homomorphic acceleration via compression of the key-value cache for disaggregated llm inference. In *Proceedings of the ACM SIGCOMM 2025 Conference*. 1245–1247.
 - [92] Jiawei Zhao, Zhenyu Zhang, Beidi Chen, Zhangyang Wang, Anima Anandkumar, and Yuandong Tian. 2024. GaLore: Memory-Efficient LLM Training by Gradient Low-Rank Projection. In *Forty-first International Conference on Machine Learning*. <https://openreview.net/forum?id=hYHsrKDiX7>
 - [93] Jiawei Zhao, Zhenyu Zhang, Beidi Chen, Zhangyang Wang, Anima Anandkumar, and Yuandong Tian. 2024. GaLore: memory-efficient LLM training by gradient low-rank projection. In *Proceedings of the 41st International Conference on Machine*

Learning (ICML'24). JMLR.org, Article 2528, 23 pages.

- [94] Yanli Zhao, Andrew Gu, Rohan Varma, Liang Luo, Chien-Chin Huang, Min Xu, Less Wright, Hamid Shojanazeri, Myle Ott, Sam Shleifer, et al. 2023. Pytorch fsdp: experiences on scaling fully sharded data parallel. *arXiv preprint arXiv:2304.11277* (2023).

A A Faster Solution for Variable Bitwidth Allocation

We propose a faster solution that dynamically maintains and adjusts an approximate value of $T_{a,b}$ across training rounds, assuming that there are at most three possible bitwidths. We explain the algorithm for the setup used in DYNAMIQ's prototype, where the allowed bitwidth is $W = \{2, 4, 8\}$. To accelerate the calculations, we avoid sorting the array of F_j values (which is needed by the algorithm described in Section 3.2) and instead calculate how many bits q_j each super-group j is quantized by the following equation:

$$q_j = 2^{\text{clamp}\left(\left\lceil \log_2\left(\frac{4}{\log_2(512/17)} \log_2 F_j + u\right) \right\rceil, 1, 3\right)}.$$

Recall that the thresholds need to satisfy $T_{2,4} = 17/512 \cdot T_{4,8}$ and the bandwidth constraint, which is $S \cdot \sum_j q_j \leq d \cdot \bar{b}$. Here, $\bar{b}$ represents the overall bitwidth budget minus the per-entry bandwidth used for transmitting the metadata, etc.

We now explain why this equation respects these constraints and how we determine u . For a constant u , let

$$z_j = \frac{4}{\log_2(512/17)} \log_2 F_j + u.$$

Observe that $q_j = 2$ if $z_j < 4$, $q_j = 4$ if $z_j \in [4, 8)$ and $q_j = 8$ if $z_j \geq 8$. Since $T_{a,b}$ is defined as the threshold for which super-groups j with $F_j \geq T_{a,b}$ are assigned with at least a bits and super-groups with $F_j \leq T_{a,b}$ are assigned with at most b bits, we have $T_{2,4} = F_j \implies z_j = 4$ and $T_{4,8} = F_j \implies z_j = 8$. This yields the following equations:

- $4 = \frac{4}{\log_2(512/17)} \log_2 T_{2,4} + u$
- $8 = \frac{4}{\log_2(512/17)} \log_2 T_{4,8} + u$
- $S \cdot \sum_j q_j \leq d \cdot \bar{b}$

Our goal is thus to adjust u via a binary search to satisfy the bitwidth constraint. That is, we decrease u if the calculated $\sum_j q_j > d \cdot \bar{b}$ in the current round, and vice versa. With u established, we can determine q_j .

B Analysis with Different All-reduce Topologies

Unlike many prior compression schemes [36, 56, 87, 90] that are designed for parameter-server-based aggregation [55], DYNAMIQ naturally supports different multi-hop all-reduce topologies. That includes two well-known all-reduce topologies, namely ring all-reduce [1] and butterfly all-reduce [78] (also known as the recursive doubling [78], as illustrated in Figure 13). We remark that compared with ring all-reduce, butterfly all-reduce typically achieves lower tail latency in large-scale DDP training systems [72].

We further observe that deploying DYNAMIQ to butterfly all-reduce also improves *scalability* with respect to its compression error as n grows larger. The intuition is that the compression error at each hop is proportional to the values of the partial sum being transmitted, which is in turn proportional to the size of the corresponding subtree if gradients on different workers follow the

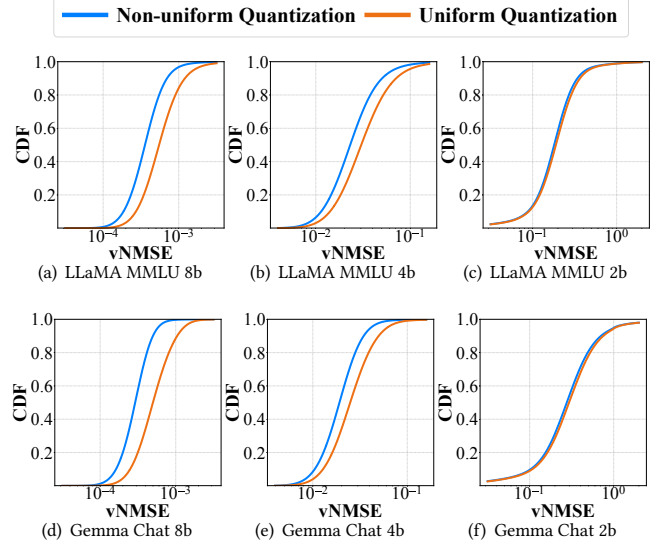


Figure 12: Compression error comparisons measured in vNMSE [81] per super-group between non-uniform quantization and uniform quantization. Each super-group uses 2, 4, or 8 bits per coordinate, and we plot the CDFs for each bit width separately.

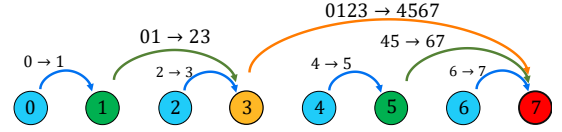


Figure 13: Butterfly all-reduce topology for a specific gradient chunk. Each color represents concurrent transmissions.

same distribution. Figure 13 illustrates this claim: worker 3, with a subtree size of 4, compresses the partial sum of worker 0 ~ 3's gradients and transmits it to worker 7, which holds the partial sum of workers 4 ~ 7.

We now heuristically analyze the compression error with ring all-reduce and butterfly all-reduce. For this analysis, we use the sum of the expected mean squared error (MSE) at each worker. We assume that the gradient data $X_{i,j}[k]$ indexed at k of the j 'th super-group at worker i is bounded by $M = \max_{i,k} |X_{i,j}[k]|$. It can be derived that the MSE for compressing the partial sum gradient $s_{i,j}$ at worker i is bounded by $\text{MSE} \leq \epsilon S \max_k |s_{i,j}[k]|^2$. We note that $\max |s_{i,j}[k]| \leq M |\text{subtree}(i)|$ where $|\text{subtree}(i)|$ is the subtree size rooted at worker i . Thus, with ring all-reduce, the expected worst-case MSE can be bounded by

$$\text{MSE} \leq \sum_i \epsilon S i^2 M^2 = O(\epsilon S M^2 n^3),$$

while that of butterfly all-reduce is

$$\text{MSE} \leq \sum_{l \leq \log n} \epsilon S (M 2^l)^2 \cdot (n/2^l) = O(\epsilon S M^2 n^2).$$

That is, our upper bound on the MSE for the butterfly is a factor of n less than that for the ring.

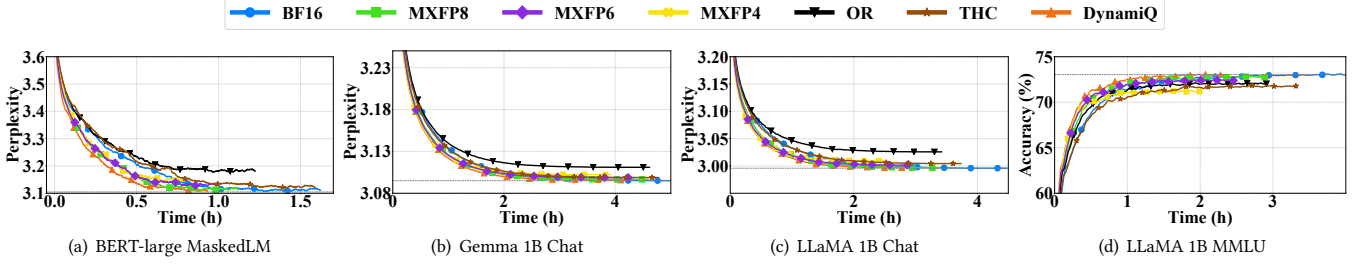


Figure 14: Zoomed-out TTA curves for LLM training and fine-tuning on an 8-GPU/4-worker testbed using ring all-reduce.

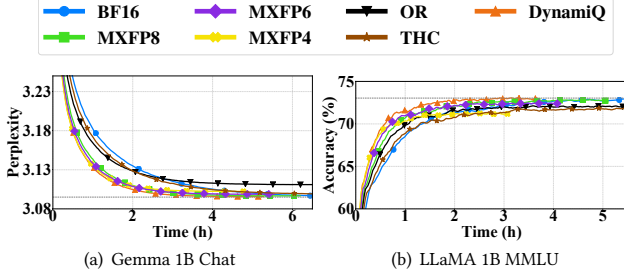


Figure 15: Zoomed-out TTA curves over a shared network.

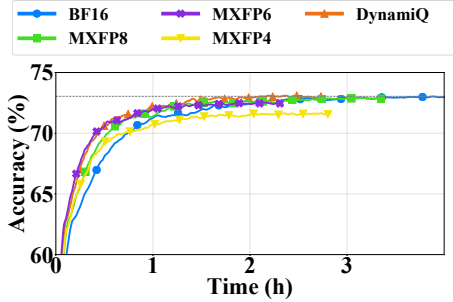


Figure 16: Zoomed-out TTA (LLaMA 1B MMLU) for butterfly all-reduce.

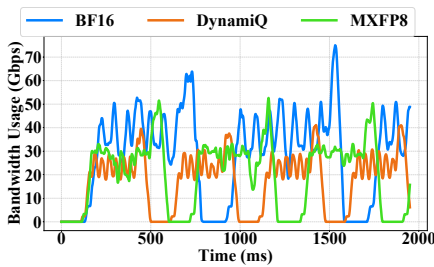


Figure 17: Bandwidth usage over time for the LLaMA 1B MMLU workload with ring all-reduce.

C Additional Experimental Setup

Adaptation of Omnireduce to ring all-reduce. As noted, Omnireduce (OR) was originally tailored for parameter-server architectures. Specifically, OR employs chunked Top- k compression, where each worker selects and aggregates its local top- k gradient chunks. In a single-hop parameter-server architecture, this is easily achieved by having workers send their local top- k chunks directly to the server. However, in multi-hop all-reduce, the local top- k

chunk indices can differ across workers; consequently, the number of aggregated chunks in an intermediate hop can exceed k . This leads to increased communication overhead, as more than k chunks may need to be transmitted per hop. To address this, we propose an adaptation that computes the union of indices appearing in at least one worker's local top- k selection. We refer to these as the global top- K chunks, where $K/n_{\text{chunks}} = b/16$ (i.e., both equal to the desired compression ratio). Given a fixed K , it is challenging to directly determine the required local k because it varies dynamically with the gradient distribution. We therefore propose a heuristic to approximate k . In each round t , given k_t , we compute the actual number of global chunks, K'_t , obtained from the union of workers' local top- k_t chunks. We then use the ratio K/K'_t to adjust k_{t+1} so that K'_t matches the target K . We update k_{t+1} according to the following momentum-based rule, where $0 \leq \gamma \leq 1$ represents the momentum (set to $\gamma = 0.8$ in our experiments):

$$k_{t+1} = \gamma k_t + (1 - \gamma) \cdot (K/K'_t) \cdot k_t \quad (1)$$

Adaptation of microscaling floating-point compression (MXFPX) to all-reduce.

As the specification of microscaling floating-point compression formats (i.e., MXFP8, MXFP6, MXFP4) [7] does not define the summation arithmetic required for all-reduce, we follow the FP8-LM [68] implementation to adapt MXFPX. The algorithm maintains a parameter μ , initialized as n , which controls the scaling factors for quantizing BF16 gradients to MXFPX. In each round, we first compute, on each worker i and for each chunk j , the maximum absolute value $m_{i,j}$ of the gradient chunk. These values are all-reduced across workers to obtain the global maximum $gm_j = \max_i(m_{i,j})$. We then determine the global scaling factor of the chunk as $s_j = \mu \cdot gm_j$, such that the original gradient $g_{i,j}$ is quantized as $g'_{i,j} = (g_{i,j}/s_j) \cdot \text{FPX_MAX}$, where FPX_MAX is the largest value representable by MXFPX. The quantized $g'_{i,j}$ is then aggregated via all-reduce. The choice of μ is critical: a smaller μ leads to more overflows as $g'_{i,j}$ becomes larger, while a larger μ causes underflows. We thus adopt the automatic scaling technique proposed in FP8-LM to dynamically update μ . If the overflow ratio exceeds a threshold ϵ , μ is updated to 2μ in the next training step. Conversely, if the overflow ratio remains smaller than γ , we decrease μ to $\gamma\mu$, where $0 < \gamma < 1$ and γ is chosen to be close to 1.

D Additional Evaluation Results

Zoomed-out end-to-end TTA curves. While Figures 5, 8, and 9 in the main text present zoomed-in versions of the time-to-accuracy (TTA) curves for ring and butterfly all-reduce, we provide the full zoomed-out versions in Figures 14, 15, and 16, respectively. These full-scale plots illustrate how each method progresses from an

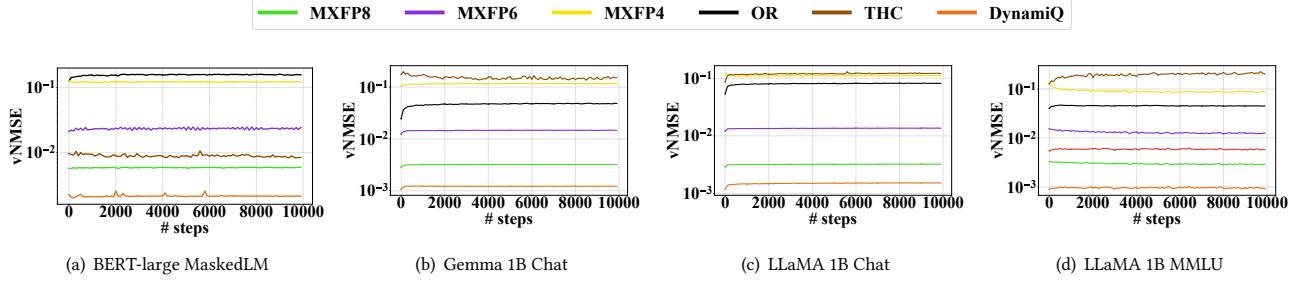


Figure 18: vNMSE comparison over different workloads on our 8-GPU/4-worker testbed with ring all-reduce.

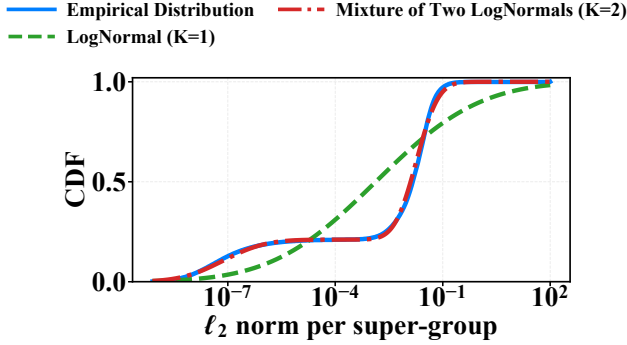


Figure 19: Empirical distribution of per-super-group ℓ_2 norms compared with CDFs fitted using mixtures of LogNormal distributions. Here for $K = 1$: $\pi_1 = 1, \mu_1 = -5.11, \sigma_1 = 3.14$ and for $K = 2$: $\pi_1 = 0.79, \pi_2 = 0.21, \mu_1 = -3.99, \mu_2 = -16.41, \sigma_1 = 1.22, \sigma_2 = 4.63$.

initial low accuracy toward the BF16 baseline (depicted by dashed horizontal lines) over time. Furthermore, the full versions confirm that the converged accuracies shown in the zoomed-in plots remain stable over extended periods without further improvement.

Bandwidth usage over time. Figure 17 illustrates bandwidth usage over time during the training of LLaMA 1B on MMLU. It distinguishes between active computation during the forward pass and network communication during overlapped backpropagation and gradient aggregation. The curves are periodic, with each period representing one training round. This clearly demonstrates that DYNAMIQ effectively reduces the time per training round (improving throughput) by minimizing communication overhead: while the computation intervals remain consistent across BF16, DYNAMIQ, and MXFP8, the communication intervals are significantly shortened, indicating faster aggregation.

Compression error curves over training steps. We also examine the evolution of vNMSE as training progresses, plotted in Figure 18. The results show that the vNMSE for DYNAMIQ and most baselines remains relatively steady, even as gradient distributions evolve during model convergence. Notably, Omnireduce (OR) shows an increase in vNMSE early in training. This suggests that gradient sparsity decreases as training progresses, making OR’s fixed-ratio sparsification less effective and leading to higher compression errors.

E Synthesizing Data for Large-scale Simulation

We generate the magnitude $M_{\mathcal{G}}$ of each super-group $\mathcal{G}$ by fitting a mixture of LogNormal distributions to the empirical distribution of super-group ℓ_2 norms shown in Figure 1(c). Specifically, for a mixture with $K \in \mathbb{N}^+$ components, we define

$$\phi(x; \theta) = \sum_{k=1}^K \pi_k \text{LogNormal}(x \mid \mu_k, \sigma_k), \quad (2)$$

where

$$\theta = \left(\{\pi_k\}_{k=1}^K, \{\mu_k\}_{k=1}^K, \{\sigma_k\}_{k=1}^K \right),$$

$\pi_k \geq 0, \sigma_k > 0$, and $\sum_{k=1}^K \pi_k = 1$. Thus, the mixture has $3K - 1$ free parameters.

Because the empirical magnitudes span several orders of magnitude, we fit θ by minimizing the squared discrepancy between the empirical and fitted CDFs over a logarithmic scale:

$$\min_{\theta} \int_{\log x_{\min}}^{\log x_{\max}} [\hat{F}(e^t) - F_{\theta}(e^t)]^2 dt, \quad (3)$$

where $\hat{F}$ is the empirical CDF, F_{θ} is the CDF corresponding to $\phi(\cdot; \theta)$, and $[x_{\min}, x_{\max}]$ is the observed range of magnitudes.

Figure 19 compares fits obtained using one and two LogNormal components. The two-component mixture closely reproduces the empirical distribution, so we use $K = 2$ and sample $M_{\mathcal{G}} \sim \phi(\cdot; \theta)$ when generating the synthetic data.